

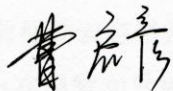
分类号	<u>TP391</u>	密级	<u>公开</u>
UDC	<u>004</u>	学位论文编号	<u>D-10617-30852-(2019)-02004</u>

重庆邮电大学硕士学位论文

中文题目	<u>基于分级检查点技术的移动云容错策略</u>
英文题目	<u>Fault Tolerance Strategy of Mobile</u> <u>Cloud Based on Hierarchical Checkpoint</u>
学 号	<u>S160231004</u>
姓 名	<u>曹启彦</u>
学位类别	<u>工程硕士</u>
学科专业	<u>计算机技术</u>
指导教师	<u>何利 教授</u>
完成日期	<u>2019 年 5 月 14 日</u>

独 创 性 声 明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包含他人已经发表或撰写过的研究成果，也不包含为获得 重庆邮电大学 或其他单位的学位或证书而使用过的材料。与我一同工作的人员对本文研究做出的贡献均已在论文中作了明确的说明并致以谢意。

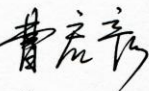
作者签名：

日期：2019 年 5 月 30 日

学位论文版权使用授权书

本人完全了解 重庆邮电大学 有权保留、使用学位论文纸质版和电子版的规定，即学校有权向国家有关部门或机构送交论文，允许论文被查阅和借阅等。本人授权 重庆邮电大学 可以公布本学位论文的全部或部分内容，可编入有关数据库或信息系统进行检索、分析或评价，可以采用影印、缩印、扫描或拷贝等复制手段保存、汇编本学位论文。

（注：保密的学位论文在解密后适用本授权书。）

作者签名：

日期：2019 年 5 月 30 日

导师签名：

日期：2019 年 5 月 31 日

摘要

移动云系统技术的快速发展,使得其复杂度日益提高,系统故障概率急遽增大。这些故障不仅会对服务提供商和用户造成巨额经济损失,还可能导致严重的灾难性事件。因此,移动云系统的容错性能成为了一个重要的研究课题,其中,基于分级检查点技术的容错策略成为了当前移动云容错领域的主要技术。

本文主要对分级检查点技术进行了研究,已经取得的研究成果主要包括:

1. 针对当前移动云容错开销较大的问题,本文依据随机更新回报理论,结合系统故障概率分布函数和累积分布函数,提出了一种基于分级检查点技术的容错算法,对分级检查点设置频率进行了动态确定。通过大量实验验证了本文算法不仅可以对不同类型故障进行针对性容错,同时可以有效的降低系统优化容错的开销,为移动云容错策略提供了一种新的解决方案。

2. 为提高移动云系统的服务质量(Quality of Service, QoS),分级检查点必须具有高可用性和高可靠性。因此,本文设计了一种基于超图覆盖的分级检查点存储策略。首先,运用本文提出的容错算法,获取到分级检查点的时间序列。然后,基于超图理论对分级检查点的存储位置进行决策。通过实验比较证明,本文提出的策略在存储任务的执行时间开销和均衡系统中各数据节点的负载方面有明显优势。

最后,本文设计并实现了基于 Python Django 框架的分级检查点容错系统,对本文的所有研究成果进行了显性验证,为移动云系统的容错技术提供了一种新的现实方案。

关键词: 移动云, 容错, 分级检查点技术, 超图存储

Abstract

With the rapid development of mobile cloud technology, the complexity of mobile cloud system is increased, and the probability of system failures is rised dramatically. These failures not only cause the huge economic loss of service providers and users, but can also lead to serious catastrophic events. Therefore, both the research community and the government-user agencies attach great importance to the study of fault-tolerance on mobile cloud. Whereas, fault-tolerance strategies based on hierarchical checkpoint technology have become the crux technique in the current mobile cloud fault-tolerance field.

The thesis mainly focuses the technology of hierarchical checkpoint. The primary achievements of the thesis include as following:

1. In view of the fault-tolerance overhead of mobile cloud system. According to a random renewal reward theory, integrating with the probability density function (PDF) and the cumulative distribution function (CDF) of the system, a fault tolerance algorithm based on hierarchical checkpoint is proposed to optimize the large fault tolerance cost of mobile cloud and dynamically determines the frequency of the hierarchical checkpoint. The numerical experiments prove the algorithm not only can cope with diffident species failures of the system, but also optimize the cost of fault tolerance, which provides a new solution for fault tolerance in mobile cloud.

2. In order to improve the service quality of mobile cloud systems, hierarchical checkpoints must be high availability and high reliability. Therefore, a hierarchical checkpoint storage strategy based on supergraph coverage is designed in this thesis. Firstly, the fault tolerant algorithm proposed in this thesis is used to obtain the time series of hierarchical checkpoints. Then, based on hypergraph theory, the storage location of the hierarchical checkpoint is decided. Experiments show that the proposed strategy has obvious advantages in the execution time overhead of storage tasks and load balance of data nodes in the system.

Finally, a hierarchical checkpoint fault-tolerant system based on the Python Django framework is designed and implemented. The system can explicitly verify all the research results in this thesis and provide a new realistic solution for the fault-tolerant technology of mobile cloud system.

Keywords: mobile cloud, fault-tolerance, hierarchical checkpoint, hypergraph coverage storage

目录

图录	VI
表录	VIII
第 1 章 绪论	1
1.1 研究背景及意义	1
1.2 国内外研究现状	3
1.3 本文主要工作	7
1.4 论文组织结构	7
第 2 章 相关理论基础	9
2.1 检查点技术	9
2.1.1 检查点技术原理	9
2.1.2 检查点技术分类	11
2.1.3 分级检查点技术	11
2.2 超图数学理论	16
2.2.1 超图定义	16
2.2.2 海明距离	16
2.3 本章小结	17
第 3 章 基于分级检查点技术的容错模型	18
3.1 分级检查点模型	18
3.1.1 随机更新回报理论	20
3.1.2 分级检查点频率函数	20
3.2 算法与实验	23
3.2.1 算法性能分析	23
3.2.2 实验环境及结果	25
3.3 本章小结	28
第 4 章 基于超图覆盖的分级检查点存储策略	30

4.1 移动云中分级检查点的超图覆盖存储策略	31
4.1.1 超图覆盖方法	31
4.1.2 分级检查点存储策略	32
4.2 超图覆盖存储算法	34
4.2.1 算法描述与性能分析	34
4.2.2 实验结果及分析	36
4.3 本章小结	41
第 5 章 分级检查点容错系统的设计与实现	42
5.1 系统设计	42
5.1.1 功能设计	42
5.1.2 模块设计	43
5.1.3 数据库设计	44
5.2 系统实现与界面展示	45
5.3 本章小结	47
第 6 章 总结与展望	49
6.1 总结	49
6.2 未来的工作	50
参考文献	51
致谢	57
攻读硕士学位期间从事的科研工作及取得的成果	58

图录

图 1.1 检查点技术	3
图 2.1 检查点工作原理示意图	10
图 2.2 移动云系统结构模型	12
图 2.3 两级检查点技术示意图	12
图 2.4 两级增量检查点技术示意图	14
图 2.5 多级检查点技术示意图	15
图 2.6 三维空间中两个顶点间距离示意图	17
图 3.1 分级检查点模型	19
图 3.2 基于分级检查点技术的容错模型算法伪代码	24
图 3.3 设置检查点时间开销	26
图 3.4 发生故障后重新计算时间开销	27
图 3.5 故障恢复时间开销	27
图 3.6 移动云系统总额外时间开销	28
图 4.1 超图覆盖模型示意图	32
图 4.2 基于超图覆盖的分级检查点存储策略的功能模块图	34
图 4.3 基于超图覆盖的分级检查点存储算法伪代码	35
图 4.4 系统规模为 128 节点时存储任务平均执行时间	36
图 4.5 系统规模为 128 节点时各数据节点剩余存储量	37
图 4.6 系统规模为 256 节点时存储任务平均执行时间	38
图 4.7 系统规模为 256 节点时各数据节点剩余存储量	39
图 4.8 系统规模为 32 节点时存储任务平均执行时间	40
图 4.9 系统规模为 32 节点时各数据节点剩余存储量	40
图 5.1 分级检查点容错系统结构图	43
图 5.2 管理员用户登录界面	46
图 5.3 管理员用户注册界面	46

图 5.4 分级检查点管理界面	46
图 5.5 分级检查点存储位置界面	47
图 5.6 系统参数列表详情界面	47

表录

表 3.1 分级检查点模型参数设计表	19
表 3.2 实验配置信息	25
表 3.3 数据节点的相关参数设置	26
表 5.1 管理员用户信息表(UserInfo).....	44
表 5.2 数据节点信息表(DataNodeInfo).....	44
表 5.3 分级检查点信息表(HCheckpointInfo).....	44
表 5.4 分级检查点存储信息表(CkptStorage).....	45

第 1 章 绪论

1.1 研究背景及意义

自 2010 年，谷歌首席执行官埃里克施密特在采访中将移动云计算（Mobile Cloud Computing, MCC）描述为“基于云计算服务的发展，移动电话将变得越来越复杂，并演变为便携式超级计算机”^[1]以来，至今已将近十载，MCC 已取得丰硕发展成果。移动云中的各类资源被虚拟化并分配到一组数量庞大且地理位置分布的计算机中，使得它们能够为移动用户、网络运营商及移动云计算提供商提供丰富的云服务^[2-4]。除此之外，MCC 作为一种新颖的计算模式，它将先前基于移动设备的密集计算、数据存储和海量信息处理都卸载到“云”中执行，从而无需再追求拥有强大的设备配置，如 CPU 速度、内存容量等，同时还为移动用户提供多种好处，例如延长电池寿命、降低移动端处理负荷等^[5]。近十年，MCC 也正是因其能够提供平稳且有效的云服务而受到学术界和工业界研究团体的广泛关注。

随着智能设备的蓬勃发展，越来越多的移动应用程序不断涌现并吸引了人们的广泛关注，如基于人脸识别、自然语言处理、互动游戏以及增强现实等技术的移动应用。然而，这些移动应用通常资源匮乏、需要密集型计算且能耗高。同时，由于物理尺寸的限制，移动设备通常受限于资源和电池寿命。可见，资源匮乏的应用程序和资源受限的移动设备之间的紧张关系成为了移动云平台开发的重大挑战。移动云计算成为解决这些挑战的有效方法，它通过无线网络接入的方式，架构于资源丰富的云基础设施之上，从而将移动应用程序中密集型计算任务和高能耗任务卸载到云端服务器执行，使得移动设备具有了满足移动应用的资源需求的能力^[6]。因此，移动云不仅要为企业和个人提供个性化的按需自助服务和可扩展性，还需要能提供高性能、高可靠且价格低廉的云基础设施、平台以及服务^[7]。在这样的背景下，移动云平台中持续几毫秒的响应时间增加或故障中断都将造成服务提供商和用户难以承受的巨额损失^[8]。2018 年 2 月 15 日，谷歌云数据库发生故障，导致众多流行的在线互动游戏崩溃，诸如 Pokemon Go、Snapchat 等受到严重影响，游戏

玩家在社交平台上的抱怨非常严重^①。2018 年 6 月 27 日下午，阿里云发生故障，导致新浪微博应用程序出现访问故障及注册短信接口全部失效问题，大量网友发表不满情绪且本次阿里云平台出现访问故障问题上微博热搜榜^②。2017 年 1 月 31 日，GitLab 由于数据库目录被误删导致中断提供服务 18 小时，本次故障影响了大约 5000 个项目、5000 条评论及 700 个新用户账户^③。由此可见，移动云服务质量 (Quality of Service, QoS) 成为移动用户和云服务提供商所共同关注的焦点问题。可靠性是从用户视角反映云 QoS 的关键评价指标。高效率和高数据安全的移动云系统资源管理与容错是保证服务可靠性的有效方法，尤其是根据移动云系统的实时状态自适应地设置分级检查点是备受青睐的容错技术之一。

为了对云资源进行合理化管理，提高移动云系统可靠性的同时尽可能不降低其性能，理论计算机科学和数学领域的科研人员提出并研究了大量的关于面向可靠性的移动云资源容错模型和方法。其中，检查点技术是目前被广泛应用于为移动云提供容错功能和故障恢复的手段。它是由移动设备通过无线网络接入云服务器，将移动应用程序中密集型计算任务卸载到云服务器中。在这些卸载到云服务器中执行的密集型计算任务无故障处理期间，周期性地将云服务当前正确状态保存到稳定存储器中。如图 1.1 所示，若移动云系统在执行任务过程中发生故障，能够通过回滚到云服务器中最新的一个正确检查点时刻重新启动系统的方式，使移动云系统返回无故障状态。采用这种方式避免了移动应用程序重头开始执行，能够在一定程度上减少额外时间开销和资源浪费。然而，值得引起重视的关键问题是，在检查点技术中，设置检查点的频率非常重要。如果检查点间隔过小，则设置检查点的开销过大，这将导致较大的系统开销。相反，如果检查点间隔过大，则在发生故障的情况下，从故障中恢复并重新启动系统的时间开销将会越长。因此，在设置检查点的频率和系统总开销之间找寻一个折衷点是至关重要的。

^① 关于该案例的更多详情参见：<https://baijiahao.baidu.com/s?id=1621632547489240311&wfr=spider&for=pc>

^② 关于该案例的更多详情参见：<http://baijiahao.baidu.com/s?id=1604928711552517871&wfr=spider&for=pc>

^③ 关于该案例的更多详情参见：<http://baijiahao.baidu.com/s?id=1587731848002734811&wfr=spider&for=pc>

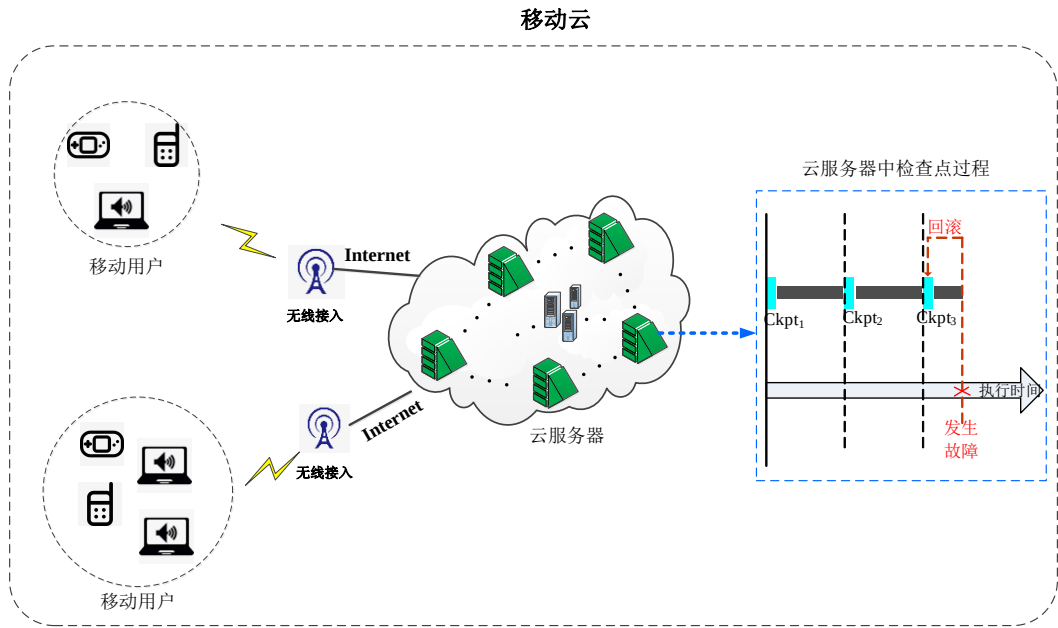


图 1.1 检查点技术

一个有效的检查点策略需要解决两个问题：如何确定检查点的设置频率和如何选取检查点的存储位置。在移动云系统中，采用合理的频率设置分级检查点可以有效的提高移动云 QoS，选取合理的检查点存储位置可以有效地保证检查点可用性及其均衡系统工作负载。鉴于此，基于分级检查点技术的移动云资源容错策略具有以下意义：

1. 合理的分级检查点设置频率可以有效地降低移动云系统从故障中恢复并重启的时间开销，保障移动云 QoS；
2. 合理的检查点存储方案能够有效保障检查点的可用性，防止系统发生故障和系统中无可用检查点的情况同时发生。同时，均衡系统中各数据节点的负载，提高系统可靠性。

1.2 国内外研究现状

成本最小化、服务质量最大化始终是移动云资源管理和容错的设计和组织的原则。为解决移动云中容错问题，研究人员主要着手于预设故障概率分布的检查点技术和无预设故障概率分布的检查点技术两个研究方向，展开了相应的理论研究和实践探索，并设计出了一系列云环境下的检查点模型。

对于预设故障概率分布的检查点技术，系统故障概率大多被预设服从泊松分布，以最小化因故障造成的额外时间开销、减少系统重新计算时间开销、降低系统恢复开销等为目的，采用不同算法确定检查点的设置频率。

J. Young^[9]在早期的检查点策略研究中做出杰出贡献，他所设计的检查点算法首先将系统故障概率预设为泊松分布，并结合系统平均故障时间和故障损失时间推算出最佳设置检查点的时间间隔，该经典算法有效减少了因故障而导致的额外任务时间开销。同时，该检查点策略也成为了检查点策略研究的里程碑。文献^[10-12]中，采用 Markov 模型中各种状态及其状态转换概率来刻画云系统中任务执行或发生故障的事件，但仍然将故障概率预设为泊松分布来获得最佳检查点间隔。为了寻求容错成本和云系统可靠性的均衡，I. Dimitriou^[13,14]使用重试队列构建检查点容错模型，在一定程度上优化了云平台容错成本及系统可靠性。然而，用户的需求日益趋于多样化，并且期望云平台能够提供高可靠性、高稳定性和价格低廉的服务。为满足这些需求，文献^[15,18]中介绍了云系统可靠性优化方法，这些方法采用点对点技术在云集群中边缘数据中心上设置检查点，以此实现云平台的容错功能。提高云资源利用率的同时，还使得云服务提供商增加了收益。而文献^[19]中则是提出将检查点作为全局参数，虽然能够为云平台提供容错功能，但却忽略了检查点的设置和布局应该具有动态变化能力。

与预设故障概率分布的检查点技术相比，无故障概率预设的检查点技术实现更为复杂，需要兼容各种移动云服务应用场景中可能发生的各类型故障。但它也能够更加准确地、有效地对移动云资源进行管理和提高系统容错性能，减少故障恢复时间开销，为用户提供高可靠服务。目前，无故障概率预设的检查点技术主要分为单级检查点模型和分级检查点模型。文献^[20-49]对这两类检查点模型进行了研究。如下所述：

1. 单级检查点模型

在单级检查点模型中，仅仅涉及到针对一种故障概率分布的检查点，并且这类检查点被用来处理最严峻的故障情况。因此，单级检查点模型的开销通常较大。文献^[20,21]中，基于迁移的虚拟机检查点/恢复技术，采用虚拟机监视器(Virtual Machine Monitor, VMM)提供保存虚拟机状态的机制，并将检查点文件随机存储于另外一个数据节点来对检查点进行可用性保证。但这种方式在对检查点进行存储

和对故障进行恢复时需要更多的时间开销。为此, Wang Long 等人^[22]提出了一种轻量级的用于虚拟机的高频检查点和快速恢复技术, 通过写时复制、脏页预测和就地恢复的方式来最小化检查点开销并加速系统故障恢复。文献[23,24]中所提出的单级检查点策略均存在巨大的检查点开销。而文献[25]中基于协调技术的检查点策略, 要求系统中所有数据节点在进行故障恢复时相互协作共同回滚至同一系统时刻。这种检查点技术也称为全局检查点技术, 它意味着在执行检查点期间所有数据节点的工作都需要暂停, 这将导致大量系统时间开销浪费。但另一种非协调检查点技术^[26]则不会在各数据节点之间施加任何协调操作。由于检查点操作并未在所有数据节点中同时执行, 倘若找不到合适的全局检查点时, 系统中各数据节点所执行的工作都必须回滚至初始状态, 即有可能出现多米诺骨牌效应问题。除此之外, 还有一种基于通信引发的检查点技术^[27], 通过在数据节点之间搭载一些特殊信息来防止发生多米诺骨牌效应。G. Berkin 等人^[28]还提出在数据节点地理位置分布的系统中采用基于主备份复制协议的检查点, 以实现故障快速恢复和无缝转换, 在检查点回滚操作时消除恢复过程, 最大限度地减少故障导致的额外时间。这些单级检查点模型研究中大多仅着眼于检查点间隔周期的设置, 却忽略了检查点存储位置这一重要影响因素。

2. 分级检查点模型

分级检查点模型又分为了两级检查点模型、两级增量检查点模型和多级检查点模型。分层级的检查点模型旨在为不同类型的故障进行分门别类的容错。例如, 故障的类型可按其性质可以分为瞬态故障和永久态故障, 针对不同类型的故障应当采用不同的检查点对其提供更细粒度的容错功能。

为了减少设置检查点的开销和因故障导致的额外系统恢复开销, N. Vaidya^[29]提出了两级检查点恢复模型。在该模型中设置的两类检查点分别用于处理永久态故障和瞬态故障。经实验分析表明, 该两级检查点恢复模型可以实现比单级检查点模型更低的总系统额外开销。由于两级检查点模型中每种类型的故障都遵循各自独立的故障分布, 优化两级检查点模型的整体性能相对较困难。为解决这类问题, Sheng Di 等人^[30]提出一种基于两级检查点模型的优化检查点解决方案, 通过将基于模式的检查点扩展到基于区间的检查点, 综合考虑在线调度和离线调度得出最优的两级检查点模型优化方案。文献[31,32]还提出了异步的两级检查点策略, 通过

利用 NVLink 和 NVMe 进行高速互连,可修改检查点大小、带宽和计算成本等参数的最优检查点策略来减少系统容错时间开销。这种更快的互连也有助于数据更快的迁移、重叠计算的实现以及智能的选择检查点频率,使得系统能够同时利用计算资源、数据池带宽和可用存储空间。

当两级检查点恢复方案用于大规模且长时间运行的系统时,系统需要定期将有关其运行状态的大量数据传输到远程节点的可靠存储中,这样的操作对设置检查点和重新计算的时间开销影响巨大,也直接影响到系统总开销。为进一步降低系统总开销,文献[33-39]提出了基于增量检查点恢复的模型。这些模型在传统的完整检查点之间设置一些增量检查点,增量检查点中只保存恢复过程中必须使用的状态或者上一个正确状态的脏页。这种操作可以减少设置检查点的开销,从而降低系统中总的容错开销。

目前,还有许多研究人员提出采用多级检查点^[40-49]来解决检查点模型的开销问题。多级检查点允许在执行期间使用不同级别的检查点。显然,多级检查点模型比单级检查点模型更加灵活。多级检查点模型中,不同级别的检查点针对不同类型的故障进行容错,由于它们的恢复能力不同,因此它们的设置成本各不相同。

检查点技术是用于云平台中为系统提供周期性容错的功能。高效精准的检查点策略是保障云 QoS 最为关键的技术部分。目前,基于检查点技术的云系统容错资源调度主要关注的优化目标有:检查点设置频率和容错总额外开销等。虽然现有的检查点技术在设置频率方面的研究已取得丰硕研究成果。但仍然存在着亟待解决的不足之处:

1. 检查点可用性。当系统中软件和硬件组件发生永久性故障时,数据节点上检查点的可用性也遭到破坏。而现有的大多数检查点研究中,对检查点可用性的关注颇少。针对检查点可用性的保障,大多研究成果是采取为每个检查点制作多个副本并将他们随机存储到远程数据节点上。当移动云系统规模较大或应用程序长时间运行时,随着检查点数量的增多,存储这些副本所占用的时间开销和存储资源不容小觑。

2. 移动云系统各数据节点负载。由于大多数现有研究中将检查点或副本基于机架感知策略随机存储到远程数据节点中,这将可能发生系统中某些数据节点上

检查点存储任务数量很多,被检查点或其副本占用的存储资源巨大,对系统中各数据节点造成负载不均衡的问题。

1.3 本文主要工作

本文对基于分级检查点技术的移动云资源容错问题进行了深入研究,综合随机概率和超图数学理论,建立了一些解决移动云容错问题的模型并提出了一些检查点存储策略。本文主要从两个方面入手解决移动云资源的容错问题,如下:

1. 基于分级检查点技术的容错模型

鉴于移动云系统中发生故障事件的不确定性,采用由整体到部分的研究方式。首先,对整个系统中因分级检查点而造成的总额外开销进行分析;然后,根据随机更新回报理论并结合系统中故障的概率分布函数和累积分布函数,将最小化系统总额外时间开销转化为对某一个故障周期内的检查点总开销进行最小化处理,提出基于分级检查点技术的容错模型。最后,利用该模型求出分级检查点频率函数,即可动态获取系统中设置分级检查点的具体时间序列。

2. 基于超图覆盖的分级检查点存储策略

在确定分级检查点设置频率之后,为了保证检查点的可用性和可靠性,还需要对检查点及其副本进行合理地存储。现有研究成果中,大多采用基于机架感知的随机存储策略,这种存储方式对不同应用程序的检查点作统一化的数据可用性处理,在一定程度上影响了移动云系统的工作效率,与此同时,还造成了巨大的检查点存储任务时间开销。因此,本文采用基于超图覆盖的方式,对移动云系统进行超图化建模,动态地决策出检查点及其副本的合理化存储方案。

最后,本文设计并实现了分级检查点容错系统。基于 Pycharm 平台及 Python Django 框架开发了一个具有 WebUI 界面的系统。该系统能够为用户以图表的形式展示分级检查点的设置时间序列及检查点存储位置等相关数据。

1.4 论文组织结构

本论文的组织结构共分为六个章节,各章节具体组织结构和研究内容如下:

第一章：绪论。首先，本章介绍了本文的研究背景及意义，阐述对移动云系统进行基于分级检查点技术的容错处理的必要性。然后，描述了云平台中检查点技术的国内外研究现状。最后，概括了本文的主要研究工作。

第二章：相关理论基础。本章节为第三章和第四章内容的理论依据。首先，对检查点技术进行阐述。然后，对分级检查点技术进行了描述，并给出了分级检查点技术的定义和不同分级方式。接下来给出超图的数学定义。最后，描述了如何采用海明距离对超图中各顶点间的空间距离进行量化。

第三章：基于分级检查点技术的容错模型。以第二章介绍的分级检查点技术作为基础，由整体到具体，对故障率各异的移动云系统利用基于分级检查点的容错模型进行建模。首先，依托于检查点对故障进行恢复的原理表示出分级检查点模型的总额外时间开销。由于系统中的故障随机发生，故此考虑采用随机更新回报理论将最小化检查点额外总时间开销转化为求解第一个故障周期内最小的检查点额外时间开销。最后针对不同系统的故障概率，提出了分级检查点模型(HCM)用以得出分级检查点频率函数 $ckpt(t)$ ，并根据 $ckpt(t)$ 获取到检查点时间序列。

第四章：基于超图覆盖的分级检查点存储策略。本章以第三章内容提出的 HCM 模型为基础，对 Level-2 型检查点及其副本进行存储位置布局。首先，依托于超图结构对移动云进行超图化建模。然后，根据检查点原始生成节点位置及超图覆盖后的移动云拓扑结构，进行检查点及其副本存储位置的自动化决策。

第五章：分级检查点容错系统的设计与实现。本章简要地描述了分级检查点容错系统的思想及基本功能需求，并进行系统设计与模块实现。

第六章：总结与展望。本章对全文所有工作进行了总结，并指出目前工作中存在的缺陷，对未来的研究工作中值得关注的问题进行了展望。

第2章 相关理论基础

检查点技术不仅仅为移动云系统提供容错功能，同时检查点的性能也从时间效率和空间占用量两方面直接影响了移动云系统的性能。在采用检查点技术的移动云系统中，检查点设置频率和检查点文件可用性为科研人员对移动云容错能力研究提供重要依据，还可减少故障对移动云系统带来的巨额经济损失。针对于为移动云资源提供容错能力，本章和后续章节采用由浅入深、理论推导和仿真实现并重的方式，先对检查点技术进行原理分析，再对检查点设置频率进行研究。而在解决检查点文件可用性这个问题上，本章节对超图的数学理论知识进行简单概述。

2.1 检查点技术

过去十余年以来，移动计算设备已经逐步取代了台式机和大型计算机。但由于移动设备的尺寸和电池寿命在一定程度上限制了它们对诸如图像等实时地计算密集型应用的处理能力。移动云计算服务应运而生，它将移动计算设备上计算密集的应用程序动态地卸载到云服务器中执行。但无论在什么设备上进行这些密集型计算，故障都难以避免，并且无法精准地预知移动云集群中故障发生的时间和数据节点。为了提供完整的数据安全性和尽可能的减少系统故障带来的损失，现采用使用最广泛的容错技术之一——检查点技术以实现为移动云系统提供自适应容错功能。

2.1.1 检查点技术原理

检查点技术是指在移动应用程序执行过程中，采用某种策略将特定时刻的应用程序状态保存到稳定存储器中。该特定时刻的应用程序状态称为检查点，它也是任务发生故障前的一个系统稳定状态。倘若在任务执行过程中检测到系统发生故障，可以回卷到最新的一个检查点处，读取检查点恢复系统状态并继续执行直至任务成功完成。利用检查点对移动云系统进行故障恢复，避免了任务发生故障后需从头开始执行的弊端，同时还减少了移动云资源的浪费，提高了移动云服务可靠性。如图 2.1 所展示的是检查点工作原理示意图。

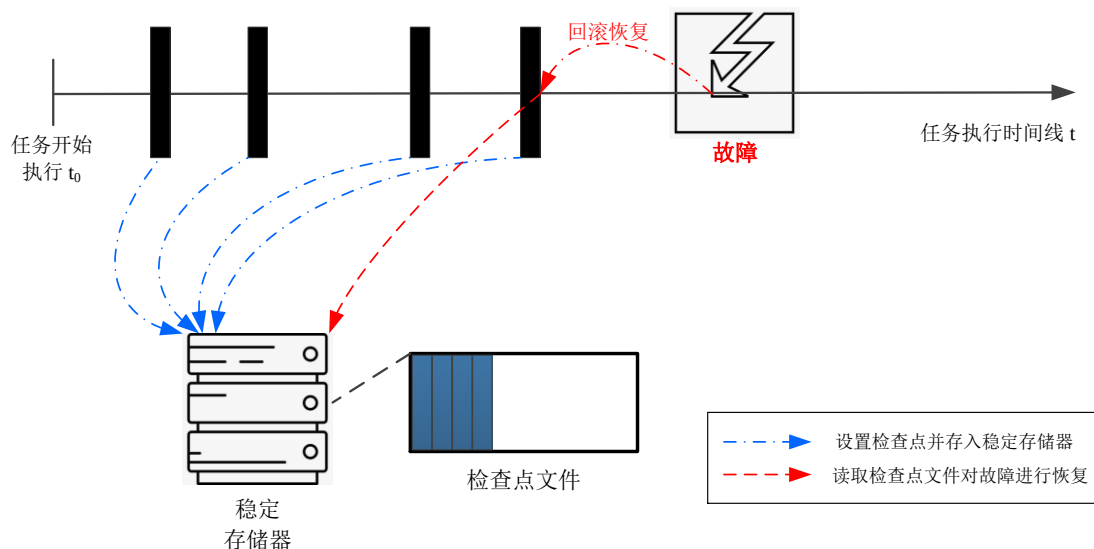


图 2.1 检查点工作原理示意图

检查点技术可为移动云系统提供容错功能，同时也意味着产生额外的系统时间开销和数据节点存储资源占用。但与无检查点的移动云系统相比，具有以下优势：

1. 减小故障损失。不采用检查点的系统中，假如任务执行过程中发生故障，则任务需要重新下发从头开始执行。而在系统中设置检查点，即可利用最新一个检查点对故障进行恢复。在一定程度上，减小了故障导致的损失。

2. 容错性强。移动云系统采用特定频率执行检查点操作，系统检测到任务发生故障时能够自动利用检查点进行故障恢复。同时，系统对检查点文件进行冗余备份并将其副本存储到另外的数据节点上。假如本数据节点中无可用检查点时，还可利用远程数据节点上的检查点副本对故障进行故障恢复。

3. 可扩展性强。检查点技术的优势之一就是可扩展性强。既可以在系统内核中加入检查点接口，也可以针对应用进程添加检查点接口。此外，还允许企业用户采用自适应算法按需为系统定制容错功能。

检查点设置频率的决策与移动云任务总执行时间开销之间存在如下的关系：若检查点设置频率高，则因设置检查点而产生的时间开销高，导致系统中任务的总执行时间会因为设置检查点的时间开销增加而增加；若检查点设置频率低，则在发生故障的情况下该任务需要重新计算的时间开销增加，由此可知，移动云任务总执行时间开销也会增加。因此，设置检查点的关键问题在于结合移动云系统的故障概率和其地理位置分布的各数据节点上处理的移动应用程序的情况，综合性考虑决策出设置检查点的频率。

2.1.2 检查点技术分类

检查点技术的分类方式并不唯一。依据其实现层次,可以分为用户级检查点和系统级检查点。用户级检查点是指由用户在其操作空间中实现检查点策略。用户级检查点的实现还可以继续细粒度划分为用户不透明实现和用户透明实现。用户不透明实现,一般是由用户决策出应用程序中需要保存的变量,这种检查点实现方式更具有灵活性。但也由于不同应用程序需要不同的检查点设置和恢复方式,用户也无法简单直观地筛选出程序中需要存储的变量,因此不透明实现方式针对不同系统的兼容性较差。而用户透明实现方式,用户只需链入检查点工具库进行编译即可。但这种方式也存在操作系统状态难以正确恢复的问题,仍然具有局限性。系统级检查点则不需要用户参与代码部分,只连接相应的检查点库即可。无需用户对检查点设置和恢复过程的细节进行了解和修改,向底层库请求检查点操作即可。在移动云环境下,适用于 Linux 操作系统的系统级检查点库的代表是 BLCR(Berkeley Lab Checkpoint-Restart)。

检查点技术还可以根据故障进行分类,通常分为单级检查点和分级检查点。单级检查点将所有故障类型进行统一处理,着重就检查点设置频率进行决策。这种方式的检查点操作比较简单,虽具有较好容错能力,但也容易导致系统开销过大。分级检查点针对不同故障做出不同处理,操作相对复杂,检查点设置频率的决策也更加困难。然而,归因于分层级对故障进行容错,检查点开销得以缩减。该方式为系统提供可靠性保障的同时,在一定程度上缩减了容错开销。

2.1.3 分级检查点技术

目前, MCC 中分级检查点技术主要两级检查点技术、两级增量检查点技术以及多层级检查点技术,旨在为复杂的移动云环境提供针对性故障容错。保障移动云 QoS、提高系统可靠性,并且尽可能减低容错开销。移动云系统结构模型如图 2.2 所示,本文后续章节将在移动云系统结构模型之上进行分级检查点模型搭建。



图 2.2 移动云系统结构模型

低容错成本和高可靠性保证是采用分级检查点技术的移动云系统中最关心的问题，检查点开销直接影响到移动云系统总任务执行时间开销。不同的检查点也因为各自的实现不同而具有不同容错开销。目前，三类主要的分级检查点技术的特点如下所述：

1. 两级检查点技术

两级检查点技术是针对系统中瞬态故障和永久性故障按照特定策略分别设置不同层级检查点进行容错处理的优质容错技术。如图 2.3 所示，两种层级的检查点都存储于数据节点的稳定存储器中。

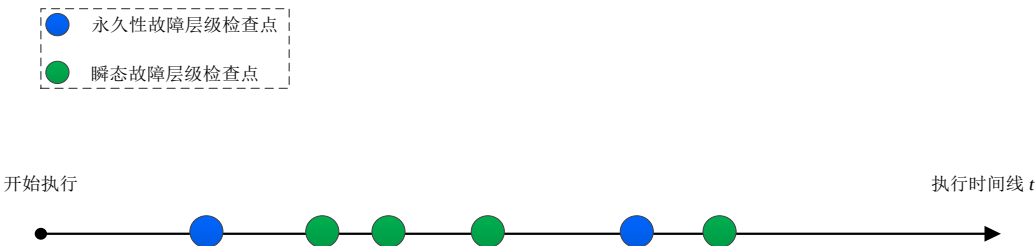


图 2.3 两级检查点技术示意图

这类两级检查点技术一般是采用内存级别的检查点对无声故障进行恢复和采用磁盘级别检查点对宕机故障进行容错。首先对两级检查点的成本时间进行建模。再结合系统历史故障数据对故障后系统的恢复开销进行最小化。动态设置两级检查点策略主要是对检查点设置频率进行研究，但普遍都具有较高的时间复杂度，例如，A. Benoit 等人^[50]所提出的针对一系列线性任务的两级检查点放置及验证算法。

该算法是一种动态编程算法，通过对检查点时间开销进行优化并返回最优解来实现对磁盘层级故障和内存层级故障提供保护和验证。对于一个任务链 $T_1, T_2, \dots, T_n$ ，首先对磁盘层级检查点开销进行建模：

$$E_{disk}(d_2) = \min_{0 \leq d_1 < d_2} \{E_{disk}(d_1) + E_{mem}(d_1, d_2) + C_D\} \quad (2.1)$$

其中， $E_{disk}(d_2)$ 表示成功执行任务 T_1 至 T_{d_2} 所需时间开销，任务 T_{d_2} 是同时在磁盘和内存中进行检查点操作和验证操作， d_1 是在 T_{d_2} 之前可能的检查点位置执行任务 T_{d_1} 所有可能的时刻； $E_{disk}(d_1)$ 表示成功执行任务 T_{d_1+1} 至 T_{d_2} 所需执行时间开销； $E_{mem}(d_1, d_2)$ 表示成功执行任务 T_{d_1+1} 至 T_{m_2} 所需执行时间开销； C_D 为设置磁盘层级检查点所需成本开销。

接下来，对内存层级检查点开销进行建模，如下公式(2.2)所示：

$$E_{mem}(d_1, d_2) = \min_{d_1 \leq m_1 < m_2} \{E_{mem}(d_1, m_1) + E_{verif}(d_1, m_1, m_2) + C_M\} \quad (2.2)$$

其中， $E_{mem}(d_1, d_2)$ 表示成功执行任务 T_{d_1+1} 至 T_{m_2} 所需执行时间开销，且在任务 T_{d_1} 之后会设置一个磁盘层级检查点，任务 T_{m_2} 之后设置内存层级检查点； $E_{verif}(d_1, m_1, m_2)$ 表示预期的执行任务 T_{m_1+1} 至 T_{m_2} 并对验证操作的执行时刻进行决策所需时间开销， C_M 为设置内存层级检查点所需成本开销。如果 $m_1 = d_1$ ，则说明在 d_1 和 m_2 之间无其他内存级检查点，从而即可得到初始化时刻的开销为 $E_{mem}(d_1, d_1) = 0$ 。

最后，为了计算两次验证之间成功执行的所有任务预期的时间开销，需要找寻最后一个磁盘检查点的时刻，最后一个内存检查点时刻，以及两次验证操作的位置 v_1 和 v_2 。于是，可以对验证操作的时间开销进行建模，如下公式(2.3)所示：

$$E_{verif}(d_1, m_1, v_2) = \min_{m_1 \leq v_1 < v_2} \{E_{verif}(d_1, m_1, v_1) + E(d_1, m_1, v_1, v_2)\} \quad (2.3)$$

其中， $E_{verif}(d_1, m_1, v_2)$ 表示预期的执行任务 T_{m_1+1} 至 T_{v_2} 所需时间开销， T_{m_1} 为最后一个内存级检查点任务， T_{d_1} 为最后一个磁盘级检查点任务，且在任务 T_{m_1+1} 和任务 T_{v_2} 之间无其他检查点； $E(d_1, m_1, v_1, v_2)$ 表示预期的执行任务 T_{v_1+1} 至 T_{v_2} 所需时间开销；若 $v_1 = m_1$ ，则表示这期间无验证操作，因此 $E_{verif}(d_1, m_1, m_1) = 0$ 。通过深入分析发现该两级检查点算法的时间复杂度较高，其通用模型的时间复杂度高达 $O(n^6)$ ，虽对系统提供了一定的容错能力，但其容错开销尚需优化。

2. 两级增量检查点技术

两级增量检查点技术是在两级检查点技术基础之上，对检查点存储的信息进行变形。两级检查点中每级检查点都是完整的检查点，而两级增量检查点中包括完整检查点和增量检查点。增量检查点中仅仅保存之前一个检查点修改的部分，并且存储与内存中。图 2.4 所示的是两级增量检查点技术示意图。

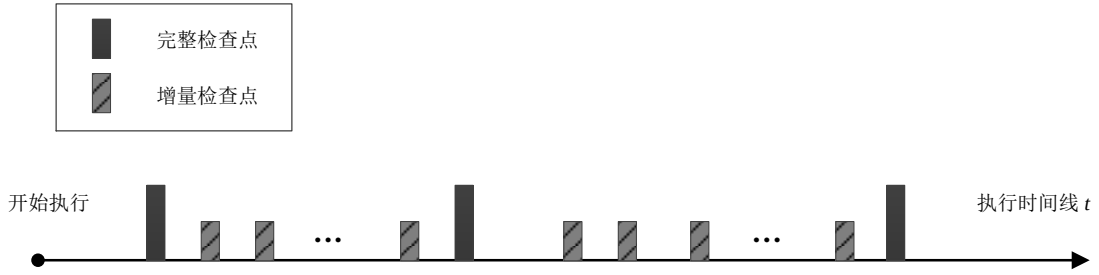


图 2.4 两级增量检查点技术示意图

例如，Chang HsungPin 等人^[35]所提出的基于增量检查点技术的无 MMU(Memory Management Unit, 内存管理单元)传感器网络容错策略。该策略设计并实施了三种增量检查点，通过分析每种增量检查点的开销确定出给定模块最佳增量检查点策略。由于应用程序模块、检查点指针和合并程序在系统中定期执行，因此在合并程序的两个相邻执行时刻之间，应用模块 m 所需开销建模如下：

$$E_A = n_e n_c E_e + n_c E_c + E_m \quad (2.4)$$

其中， n_e 表示应用程序模块在两个连续增量检查点之间运行次数， n_c 表示在两个连续合并程序之间调用检查点指针的次数， E_e 、 E_c 和 E_m 分别为应用程序模块、检查点程序和合并程序所造成的开销。然而，检查点开销又可以进一步建模为：

$$E_c = E_{c_c} + E_{c_t} + E_{c_r} + E_{c_p} \quad (2.5)$$

其中， E_{c_c} 表示执行增量检查点时 CPU 所需时间开销， E_{c_t} 和 E_{c_r} 分别表示将增量检查点数据传输到备份节点的开销和从主节点接收增量检查点数据时所需开销， E_{c_p} 表示将增量检查点数据写入内存所需开销。同样，合并程序的开销也可以进一步建模为：

$$E_m = E_{m_c} + E_{m_r} + E_{m_p} \quad (2.6)$$

其中， E_{m_c} 表示 CPU 执行合并程序时所需开销， E_{m_r} 表示从内存读取增量检查点数据时所需开销， E_{m_p} 表示将合并的检查点数据写入内存所需开销。

应用程序模块执行所需开销可以被表示为：

$$E_e = E_0 + R_e I \quad (2.7)$$

其中， E_0 表示运行原始应用程序模块无故障情况下所需开销， R_e 表示执行指令所需开销， I 表示增量检查点所需插入的指令数量。为了获取最佳增量检查点决策，将上述三部分的和最小化即可。

3. 多级检查点技术

多级检查点技术是针对系统中多种故障提供多个层级检查点进行容错操作。一般来说，对于一个具有 k 层级模式的移动云系统中，每个层级的检查点都有唯一的标识、设置频率及存储设备，也具有不同的开销和恢复能力。通常，每个层级对应与特定的故障类型，并且各类型故障具有不同故障率。直观地说，故障的频率越低，两个检查点之间的间隔也就越长：因为故障的破坏等级越高时，其故障率反而更低，则预期的重新执行时间也随之降低，从而减少检查点容错开销。图 2.5 中展示的是多级检查点技术示意图。

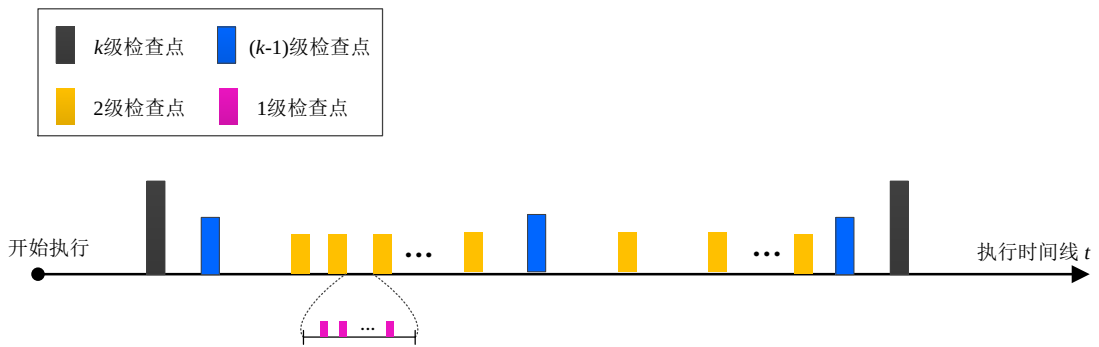


图 2.5 多级检查点技术示意图

例如，I. Jangjaimon 等人^[51]使用 Markov 模型评估增量压缩的多级并发检查点设置频率及预期的容错开销。基于 Markov 模型的多级检查点预期总开销计算模型如下公式(2.8)所示：

$$NET^2 = \sum_{i=1}^n \frac{T_{int(i)}}{t} \quad (2.8)$$

其中， NET^2 表示预期的状态周期转化时间， $T_{int(i)}$ 表示第 i 个状态间隔预期执行时间， t 表示单个任务进程执行时间。

该策略中 Markov 模型以有向图表示出状态和状态转换。在无故障情况下，使用该状态中的时间开销来表示这个状态，状态之间的边权重表示状态转换的概率。

通过这些转换概率和状态中预期的时间开销构建检查点模型，最后通过求解线性方程组获得检查点设置频率和预期的容错开销。

2.2 超图数学理论

超图是有限集合的子集系统，是离散数学中最一般的结构。早期，Erdős 和 Spencer^[52]引入了类似超图的概念。20 世纪 60 年代，“超图”这个词被正式提出来，它的基本概念和相关推论均为传统图的中定义和定理的平移和泛化。在互联网和云服务都爆炸式进化的今天，数学理论对人类社会各个领域都产生着巨大的影响，互联网领域也毫不例外地受益于数学理论提供的机遇、源泉和动力。

2.2.1 超图定义

定义 2.1 超图^[53] 设 $V = \{x_1, x_2, \dots, x_n\}$ 是一个有限集合， $\varepsilon = \{e_1, e_2, \dots, e_m\}$ 是 V 的一个子集合族，即 $\varepsilon \subseteq 2^V$ 。如果 $\forall i \in \{1, 2, \dots, m\}$ ， $e_i \neq \emptyset$ 和 $\bigcup_{i=1}^m e_i = V$ ，则 $H = (V, \varepsilon)$ 称为 V 上的一个超图。集合 V 中的元素称为顶点，集合 ε 中的元素称为边。

定理 2.1 超图顶点与边的关系^[53] 设 X 是 $n \geq 3$ 个顶点的集合， ε 是在 X 上的有 m 条边的超图， $\varepsilon = \{E_1, E_2, \dots, E_m\}$ 。假设每个 E_i 是 X 的一个真子集， X 的每个顶点都恰被包含在一条边中，则 $m \geq n$ 。

2.2.2 海明距离

从 2.2.1 章节中定义的超图可以发现，超图的一条边是由超图顶点集中某个子集中的所有顶点构成。为了对超图中顶点间距离进行量化，本章节将引入海明距离来确定超图中各顶点间的距离，进一步对超图边的长度进行明确化。

定义 2.2 海明距离 两个相等长度的字符串之间相应位置处出现不同比特的数量称为海明距离。也就是说，两个字符串 x 和 y 的海明距离可以通过异或运算求得。如果将 n 维空间的超图中的每个顶点都用 n 位二进制编码来表示，则只需要求出两个 n 位二进制编码之间的海明距离即可得到两个顶点间的欧几里得距离，进一步可以量化出超图中任意边的长度。如图 2.6 所示，一个三维空间中的顶点需

用3位码字来编码，若需得出顶点 M 与顶点 N 之间的欧几里得距离，则可使用海明距离进行求解。

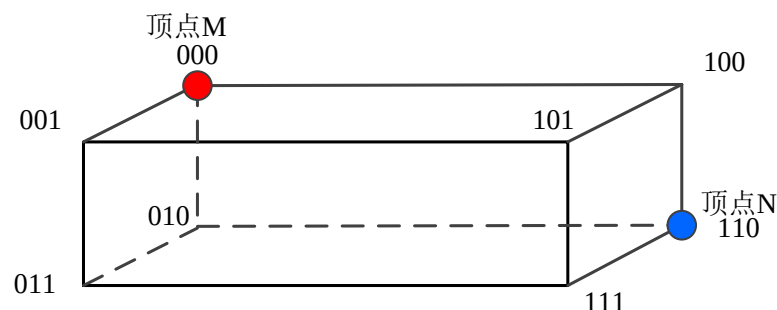


图 2.6 三维空间中两个顶点间距离示意图

从上图 2.6 中可得知，顶点 M 的 3 位二进制编码为 000，顶点 N 的 3 位二进制编码为 110。因为 $000 \oplus 110 = 010$ ，所以顶点 M 和顶点 N 之间的海明距离为 2。

2.3 本章小结

本章第一节对检查点技术进行了详细介绍，包括检查点技术原理、不同分类方式以及几种主要的分级检查点技术的特征。接着，超图的数学理论进行了讲解。最后，对量化超图中顶点间距离和超图边的长度的方法进行描述。本章内容为后续两章内容提供理论基础。

第3章 基于分级检查点技术的容错模型

随着现代数据中心的发展,移动云服务变得越来越高效和便捷,但其管理却变得愈发困难。在通过分布式共享集群实现的大多数商业平台中,用户可以获取“按需”服务。尽管如此,在如今最强大的高性能计算系统中却每天都面临着遭受一次故障的风险^[54]。移动云系统中故障大多数为软件故障引起,但也不乏由软件运行期间连续积累故障所引起的硬件故障,诸如:舍入故障,内存泄露,未释放的内存空间,存储碎片,数据节点宕机等等。归因于这些故障,系统性能将逐渐被降低,若无维护措施和容错措施,最终可能由于系统崩溃而导致惨重损失。因此,需要采用检查点技术将系统正确状态定期保存在稳定存储器上,以便在系统发生故障时利用正确检查点重新恢复系统的运作。现有的检查点容错模型大多数是将移动应用程序任务集中单个任务和整个数据节点中所有任务的故障进行统一化容错,由于不同性质的故障对系统造成的损伤程度不同,这样的统一化容错处理容易导致系统中检查点开销过大。

因此,为保证卸载到云端进行处理的移动云任务服务质量,同时尽可能地降低利用检查点进行容错造成的额外时间开销,提出了基于分级检查点技术的容错模型(Hierarchical Checkpointing Model, HCM),旨在通过合理化选取分级检查点的设置频率来降低采用分级检查点容错模型的额外时间开销,从而减少移动云系统的容错开销问题。首先, HCM 模型根据随机更新回报理论,并结合当前系统的故障概率分布函数(Probability Density Function, PDF)和故障累积分布函数(Cumulative Distribution Function, CDF),将系统中由于故障导致的总额外时间开销转化为求解某一个故障周期中故障导致的额外时间开销。接着, HCM 模型对第一个故障周期中的检查点开销进行最小化处理以获取到分级检查点频率函数。最后,通过对比实验,验证了该 HCM 模型的有效性。

3.1 分级检查点模型

在本章节中,主要针对移动云系统中任务层级和数据节点层级进行分级检查点操作,即分级检查点模型分为 Level-1 级检查点(任务层级)和 Level-2 级检查

点（数据节点层级），Level-1 级检查点中保存应用程序子任务状态并存储于内存中，Level-2 级检查点中保存系统状态并存储于磁盘中，如图 3.1 所示。这种分级检查点模型不仅可以针对不同故障类型提供细粒度的故障保护，而且能够对系统中整体的容错开销进行全局优化。此外，与传统的单级检查点模型相比，该分级检查点模型还对更多的故障概率分布类型进行兼容，并且在实践中也易于操作。

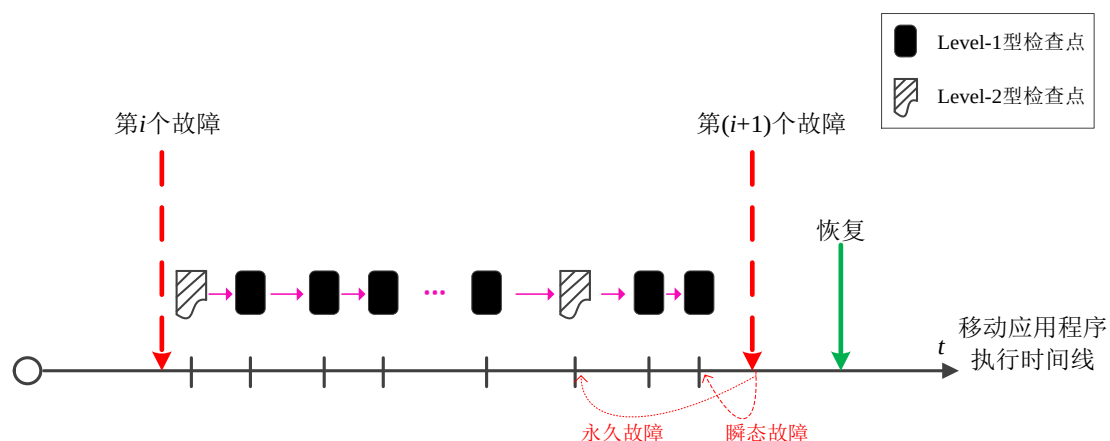


图 3.1 分级检查点模型

由于任务层级故障与数据节点层级故障拥有各自独立的故障概率分布，本文将故障按照故障的性质划分，分为瞬态故障（诸如单个计算或存储任务故障等）和永久性故障（诸如数据节点损坏等）。当瞬态故障发生时，可以通过读取最新的 Level-1 型检查点对该任务进行故障恢复，若无可用的 Level-1 型检查点也可以读取最新的 Level-2 型检查点进行故障恢复；当永久性故障发生时，移动云系统需要读取 Level-2 型检查点才能对系统进行恢复。该分级检查点模型中参数设计详细如下表 3.1 所示：

表 3.1 分级检查点模型参数设计表

参数	意义
C_1	设置 Level-1 型检查点的开销
C_2	设置 Level-2 型检查点的开销
R_1	使用 Level-1 型检查点进行故障恢复的开销
R_2	使用 Level-2 型检查点进行故障恢复的开销
ϕ	重新计算系数
$f(t)$	移动云系统中故障概率分布函数
$F(t)$	移动云系统中故障累积分布函数

3.1.1 随机更新回报理论

随机过程 $X = \{X(t), t \in T\}$ 由一组随机变量组成, 即对指标集 T 中的每个 t , $X(t)$ 是一个随机变量, 其中 t 表示时间, $X(t)$ 是过程在时刻 t 的状态^[55]。鉴于此, 移动云系统中发生故障的事件可视为一个随机过程。

故障在应用程序生命周期中随机发生, 现假设移动云系统中执行的特定应用程序在其生命周期中发生故障总次数为 Z , 故障发生的间隔时间为 $X_i, i \geq 1$, 且故障具有共同概率分布 F 。系统每发生一次故障都需重启一次, 该过程也伴随着一份额外时间开销。即每当系统发生故障并重启一次就说明系统完成了一个故障循环, 该过程符合随机更新回报过程。在移动云环境中, 随机更新回报过程可表述如下:

假设 M_i 表示第 i 次故障所导致的额外时间开销, 则 $M_i, i \geq 1$ 独立同分布, 但允许 M_i 依赖第 i 个故障周期的间隔时间 X_i 。

设 $E[M] = E[M_i], E[X] = E[X_i]$, 若 $E[M] < \infty$ 及 $E[X] < \infty$, 则

$$\lim_{t \rightarrow \infty} \frac{\sum_{i=1}^Z M_i}{t} = \frac{E(M_1)}{E(X_1)} \quad (3.1)$$

其中, $E[M]$ 表示预期的第 i 次故障所导致的额外时间开销, $E[X]$ 表示系统中第 i 个故障预期的时间间隔周期, $\sum_{i=1}^Z M_i$ 表示从系统开始执行任务直到第 i 个故障发生时刻由于故障所造成的全部额外时间开销。

由公式(3.1)所示的随机更新回报理论可知, 长时间之后的平均开销恰好是一个循环的开销除以该循环的预期时间间隔。即在移动云系统中想要获取整个移动应用程序生命周期中故障导致的额外时间开销, 可转化为获取某一个故障周期内由故障导致的额外时间开销。因此, 下文针对系统中第一个故障周期进行分级检查点频率设置。

3.1.2 分级检查点频率函数

采用分级检查点技术的移动云系统中额外时间开销一般由设置分级检查点时间开销、发生故障后重算时间开销、以及故障恢复时间开销三个部分所构成, 如公式(3.2):

$$M(T) = I(T) + R(T) + S(T) \quad (3.2)$$

其中, $I(T)$ 表示设置分级检查点的时间开销, $R(T)$ 为发生故障后重新计算的时间开销, $S(T)$ 表示故障恢复时间开销, T 为系统的各故障周期间隔时间, $M(T)$ 为设置分级检查点而造成的总额外时间开销。

假设分级检查点频率函数为 $cp(t)$, 则移动云系统中执行的移动应用程序从开始执行直到系统发生故障的故障周期 $(0, T)$ 内, 分级检查点频率函数定义为:

$$\int_{t_i}^{t_{i+1}} cp(t)dt = 1, i \geq 0 \quad (3.3)$$

其中, $t_i, (i = 1, 2, 3, \dots)$ 为第 i 个检查点放置时间, 且 $t_0 = 0$ 。

因此, 在故障周期 $(0, T)$ 内设置的分级检查点的数量 N 为:

$$N = \int_0^T cp(\psi)d\psi \quad (3.4)$$

定义 3.1 设置分级检查点时间开销 指在移动云系统中自适应地设置每个分级检查点的时间开销与对应层级检查点数量的乘积之和。在本文中, 默认系统在开始处理任何一个移动应用程序时都会先创建一个 Level-2 型检查点, 以保证初始时刻系统的可靠性。故此, 假设在故障周期 $(0, T)$ 内分级检查点序列中共有 $(\mu + 1)$ 个检查点, 则该故障周期 $(0, T)$ 中设置分级检查点的时间开销计算公式为:

$$I(T) = C_1 \frac{\mu}{\mu+1} \int_{t_i}^{t_{i+1}} cp(t)dt + C_2 \frac{1}{\mu+1} \int_{t_i}^{t_{i+1}} cp(t)dt = \frac{C_1 + \mu C_2}{\mu+1} \int_{t_i}^{t_{i+1}} cp(t)dt \quad (3.5)$$

其中, C_1 表示设置 level-1 型检查点的时间开销, C_2 表示设置 level-2 型检查点的时间开销, 且 $C_2 > C_1$; Level-1 型检查点数量在该故障周期的检查点序列中的占比为 $\frac{\mu}{\mu+1}$, Level-2 型检查点数量在该故障周期的检查点序列中的占比为 $\frac{1}{\mu+1}$ 。

定义 3.2 发生故障后重算时间开销 指当前发生故障的时刻与最新一个检查点之间的时间间隔。根据均值定理, 可以利用分级检查点频率函数表示出发生故障后重算时间开销 $R(T)$ 的近似值, 如公式(3.6)所示:

$$R(T) \approx \frac{\Phi}{cp(T)} \quad (3.6)$$

其中, Φ 为重新计算时间系数, 且 $0 < \Phi < 1$; $cp(T)$ 为该发生故障的故障周期 $(0, T)$ 中检查点的频率。依据参考文献[34]中将重新计算系数的初始值选取为0.5, 为与之进行对比实验, 本章后续将重新计算系数初始值确定为 $\Phi = 0.5$ 。

定义 3.3 故障恢复时间开销 指检索最新一个检查点并回滚到该检查点时刻的系统稳定状态所需时间开销。则移动云系统发生故障后的故障恢复开销 $S(T)$, 如公式(3.7)所示:

$$S(T) = \mu R_1 + R_2 \quad (3.7)$$

其中, R_1 和 R_2 分别为设置 Level-1 型检查点和 Level-2 型检查点的恢复时间开销, 且 $R_2 > R_1$ 。

则分级检查点造成的总额外时间开销计算公式为:

$$M(T) = \frac{C_1 + \mu C_2}{\mu + 1} \int_{t_i}^{t_{i+1}} cp(t) dt + \frac{\Phi}{cp(T)} + (\mu R_1 + R_2) \quad (3.8)$$

现假设该移动云系统的故障概率分布函数为 $f(t)$, 则分级检查点造成的总额外时间开销的期望值计算公式, 如下所示:

$$E(M) = \int_0^\infty [M(T)] \cdot f(t) dt \quad (3.9)$$

根据公式(3.8)可知, 最小化分级检查点的频率即可获取到优化的预期分级检查点额外时间开销。将公式(3.8)代入公式(3.9)可得到分级检查点总额外时间开销的期望值计算公式, 如下所示:

$$E(M) = \int_0^\infty \left[\frac{C_1 + \mu C_2}{\mu + 1} \int_{t_i}^{t_{i+1}} cp(t) dt + \frac{\Phi}{cp(T)} + (\mu R_1 + R_2) \right] \cdot f(t) dt \quad (3.10)$$

因此, 结合 3.1.1 章中所述的随机更新回报理论, 只需计算移动云系统的第一个故障周期中检查点频率即可获取到整个移动云系统生命周期的分级检查点频率函数计算公式, 如下公式(3.11)所示:

$$cp(t) = \sqrt{\frac{(\mu+1)\Phi}{\mu C_1 + C_2}} \sqrt{\frac{f(t)}{1-F(t)}} \quad (3.11)$$

其中, $F(t)$ 为移动云系统中故障的累积分布函数。

证明:

令 $q(t) = \int_0^T cp(t) dt$, 则对其求导可得 $q'(t) = cp(t)$ 。

因此, 公式(3.10)更新为:

$$E(M) = \int_0^\infty \left[\frac{C_1 + \mu C_2}{\mu + 1} q(t) + \frac{\Phi}{q'(t)} + (\mu R_1 + R_2) \right] \cdot f(t) dt$$

令 $l(q, q', t) = \left[\frac{C_1 + \mu C_2}{\mu + 1} q(t) + \frac{\Phi}{q'(t)} + (\mu R_1 + R_2) \right] \cdot f(t)$, 代入上式可将分级检查

点总额外时间开销的期望值计算公式表示为:

$$E(M) = \int_0^\infty l(q, q', t) dt$$

由变分法理论^[56]可知, 如果 $l(q, q', t)$ 存在最小值, 则其必然满足欧拉-拉格朗日公式:

$$\frac{\partial l}{\partial q} - \frac{d}{dt} \left(\frac{\partial l}{\partial q'} \right) = 0$$

然后，通过分别求解 l 中关于 q 和 q' 的偏导数可得：

$$\begin{aligned}\frac{\partial l}{\partial q} &= \frac{\mu C_1 + C_2}{\mu + 1} f(t) \\ \frac{\partial l}{\partial q'} &= -\frac{\Phi}{(q'(t))^2} f(t)\end{aligned}$$

现将上述两式代入欧拉-拉格朗日公式可得：

$$\frac{C_1 + \mu C_2}{\mu + 1} f(t) - \frac{d}{dt} \left(\frac{\Phi}{(q'(t))^2} f(t) \right) = 0$$

再上式的等号两边同时求 $(0, t)$ 区间的积分可得：

$$\frac{C_1 + \mu C_2}{\mu + 1} F(t) + \frac{\Phi}{(q'(t))^2} f(t) = B$$

其中， B 为常数。

显然， $q(0) = 0$ 。又因为 $\lim_{t \rightarrow \infty} f(t) = 0$ ，所以函数 $q(t)$ 满足 $\lim_{t \rightarrow \infty} \frac{\partial l}{\partial q'} = 0$ ，将其代入上式可求得 B 为：

$$B = \frac{C_1 + \mu C_2}{\mu + 1}$$

再将上述 B 的表达式进行代入替换则可得，

$$q'(t) = \sqrt{\frac{(\mu + 1)\Phi}{\mu C_1 + C_2}} \sqrt{\frac{f(t)}{1 - F(t)}}$$

由于 $q'(t) = cp(t)$ ，所以

$$cp(t) = \sqrt{\frac{(\mu + 1)\Phi}{\mu C_1 + C_2}} \sqrt{\frac{f(t)}{1 - F(t)}}$$

证毕。

3.2 算法与实验

3.2.1 算法性能分析

本章所提出的基于分级检查点技术的容错模型主要是按照故障性质对故障类型进行划分，并针对各类型故障提供不同层级的检查点进行区别化容错处理。然后在保证提供移动云服务可靠性前提下，通过最小化分级检查点的频率来降低移动云系统的容错开销。该模型中的总时间开销（即采用分级检查点技术导致的移动云系统总额外时间开销）由设置分级检查点时间开销、发生故障后重算时间开销、故障恢复时间开销三部分所构成：

1. 设置分级检查点时间开销：从特定移动应用程序任务集中提取各任务进程状态，再获取到设置 Level-1 型检查点的时间开销 C_1 ；从移动云系统数据节点集中提取联合执行该移动应用程序的数据节点状态，再获取到设置 Level-2 型检查点的时间开销 C_2 。采用公式(3.5)对系统设置分级检查点的时间开销进行预处理。

2. 发生故障后重算时间开销：该部分时间开销来自于对当前发生故障的时刻与最新一个检查点之间的丢失任务进行重新执行。采用公式(3.6)对系统发生故障后重算时间开销进行预处理。

3. 故障恢复时间开销：该部分时间开销产生于系统检索最新一个检查点并回滚到该检查点时刻的系统稳定状态所需时间开销。采用公式(3.7)对移动云系统故障恢复时间开销进行预处理。

基于分级检查点技术的容错模型(HCM)的算法伪代码如图 3.2 所示：

算法 3.1 HCM

输入： 系统故障概率分布函数 $f(t)$ ，系统故障累积分布函数 $F(t)$

输出： 各分级检查点放置时间 t_i ，
设置分级检查点而造成的总额外时间开销 $M(T)$

```

1:   While  $0 < t < T$  do
2:       采用公式(3.11)得到  $cp(t)$ 
3:       采用公式(3.10)得到  $\frac{1}{\mu+1_{min}}$  and  $\frac{\mu}{\mu+1_{min}}$ 
4:       采用公式(3.3)和公式(3.11)得到  $t_i$ 
5:       For  $t$  to  $t_i$  do
6:           If  $\frac{\mu}{\mu+1} < \frac{\mu}{\mu+1_{min}}$  then
7:               在当前时刻 $t$ 设置 Level-1 型检查点
8:           Else
9:               在当前时刻 $t$ 设置 Level-2 型检查点
10:          End If
11:       End For
12:       Return  $t_i$ 
13:   采用公式(3.5)计算系统总额外时间开销 $M(T)$ 
14: End While
15: If 发生故障  $failure$  then
16:      $Z++$ ;
17:     If  $failure \in Level-1 Failure$  then
18:       获取 Level-1 型检查点对系统进行重启恢复
19:     Else
20:       获取 Level-2 型检查点对系统进行重启恢复
21:     采用公式(3.8)计算系统总额外时间开销 $M(T)$ 
22:     return  $M(T)$ 
23:   End If
24: End If

```

图 3.2 基于分级检查点技术的容错模型算法伪代码

假设特定应用程序在第一个故障间隔周期 T 中, 共设置有 t_i 个分级检查点。则设置分级检查点的时间开销为 $O(\frac{C_1\mu+C_2}{\mu+1}t_i)$, 发生故障后重新计算时间开销为 $O(\frac{1}{2t_i})$, 故障恢复所花费时间为 $O(\mu R_1 + R_2)$ 。由此可见, HCM 模型总的时间复杂度为 $O(\frac{C_1\mu+C_2}{\mu+1}t_i + \frac{1}{2t_i} + \mu R_1 + R_2)$ 。由于在确定故障间隔周期中分级检查点的设置频率对 HCM 模型的容错时间开销影响度最大, 则在该周期中共设置分级检查点的数量 t_i 是 HCM 模型时间复杂度的主要影响因素。所以, HCM 模型的时间复杂度可以简化为 $O(\frac{C_1\mu+C_2}{\mu+1}t_i + \frac{1}{2t_i})$ 。

3.2.2 实验环境及结果

为了验证 HCM 模型应用于移动云系统时的容错性能, 本文选取了两级增量检查点策略与本文提出的 HCM 容错模型进行对比实验。两级增量检查点策略是 Li 等人^[34]提出的一种用于减少系统总开销的两级增量检查点恢复机制, 以优化大规模且长时间运行应用程序的系统中总系统开销为目标, 定义出三种类型的检查点, 并对全局检查点放置频率进行决策。Li 等人所提两级增量检查点策略相较于两级检查点策略而言, 在一定程度上优化了总容错开销。为验证 HCM 模型的有效性, 选择与两级增量检查点进行对比实验, 分别对比设置检查点时间开销、发生故障后重算开销、故障恢复时间开销及总容错时间开销以具体说明 HCM 模型的容错性能。

本实验基于云计算仿真平台 CloudSim 和检查点框架 BLCR 进行, 实验环境的配置信息如表 3.2 所示。

表 3.2 实验配置信息

环境	配置详情
硬件环境	Intel(R) Core(TM) i7-8700k CPU @3.70GHz
软件环境	Linux Centos 6.5 OS; JDK 1.7; IntelliJ IDEA; CloudSim3.3; BLCR

在本章中, 采用的移动应用程序数据集, 其大小随机设置在 10GB~50GB。并且设定移动云集群中最多可并行处理 8 个移动应用程序。参考 Li 等人^[34]和 Yan 等

人^[57]所设计系统中数据节点参数的设置, 本文将移动云集群中数据节点的相关实验参数设置为如表 3.3 所示。

表 3.3 数据节点的相关参数设置

参数	取值范围	默认值
数据节点数	[30,200]	128
每个数据节点核数	[4,16]	8
数据节点故障率服从指数分布 λ	$[10^{-3}, 10^{-1}]$	10^{-2}
数据节点故障率服从威布尔分布规模参数 α	2.0815	2.0815
数据节点故障率服从威布尔分布形状参数 β	0.6857	0.6875
数据节点带宽	400-800MB/s	400MB/s

当系统故障率恒定时, 故障率函数通常呈现为指数分布。另外, 威布尔分布是一种通用的可靠性分布, 主要用于电子和机械组件, 设备或者系统故障时间。在本章的 HCM 模型与两级增量检查点策略的性能对比实验中, 主要分析比较在系统故障概率分布服从指数分布和威布尔分布情况下, 设置检查点时间开销、发生故障后重算时间开销、故障恢复时间开销以及总系统额外时间开销四个部分。

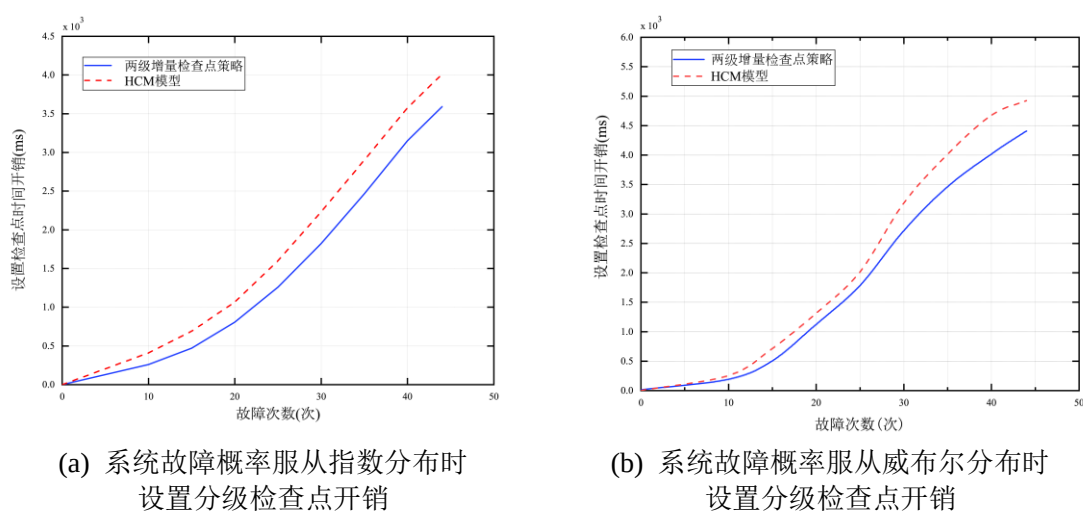


图 3.3 设置检查点时间开销

图 3.3(a)和图 3.3(b)为 HCM 模型与两级增量检查点策略设置检查点时间开销的对比实验结果。当移动云系统故障率服从指数分布时, 如图 3.3(a)所示。HCM 模型中设置分级检查点的时间开销比两级增量检查点策略中设置检查点的时间开销平均多 2.7%~4.8%。其原因在于两级增量检查点策略中, 增量检查点只保存了前一个检查点的脏页, 具有相对较快的速度。同样, 当移动云系统故障率服从威布尔

分布时,如图 3.3(b)所示。HCM 模型中设置分级检查点的时间开销仍然比两级增量检查点策略平均多出 3% ~ 5.5%。

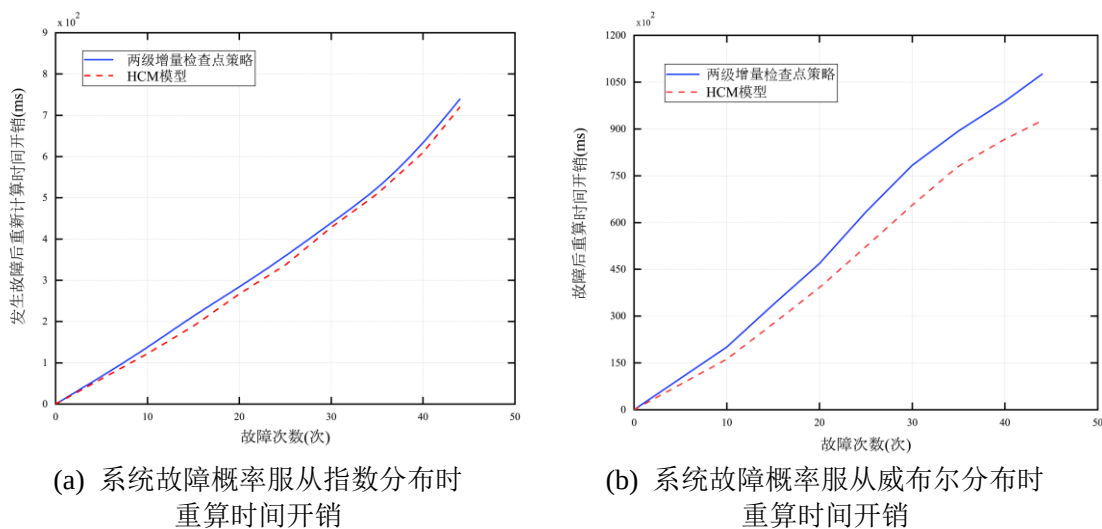


图 3.4 发生故障后重新计算时间开销

移动应用程序执行期间导致故障发生的未知影响因素非常多,即故障无法规避,其类型也无法人为干预。由图 3.4(a)和图 3.4(b)可知,系统故障率服从指数分布或者服从威布尔分布的情况下,HCM 模型中发生故障后重算时间开销相对较少。在移动云系统中,虽然发生故障后可以回滚到最新可用检查点位置对故障进行恢复,但仍然会损失发生故障时刻到检查点时刻之间的任务进度。因此,发生故障后重算时间开销越少,对减少系统容错开销和保障移动云 QoS 来说很有利。

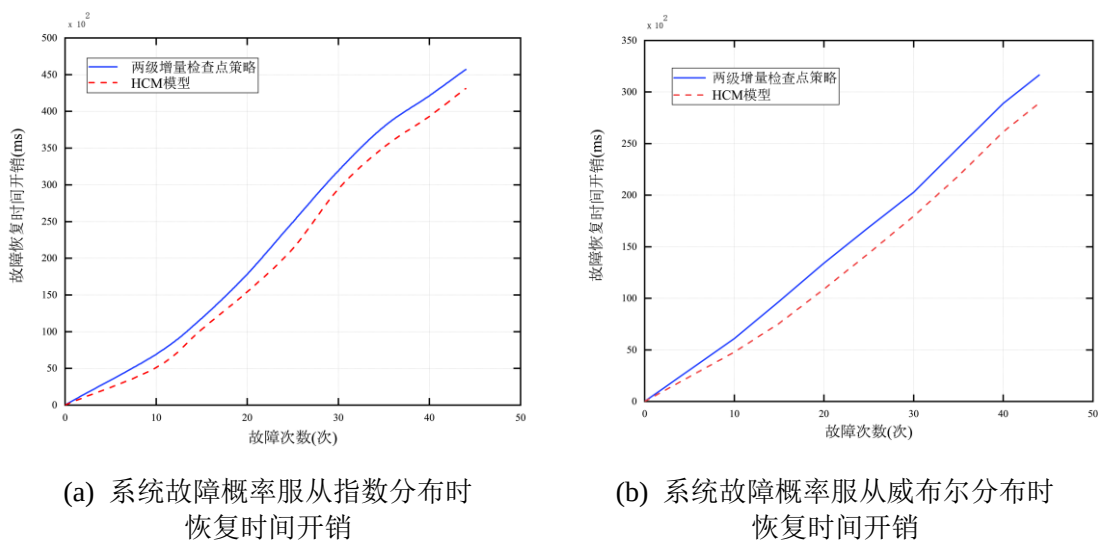


图 3.5 故障恢复时间开销

图 3.5(a)和图 3.5(b)对比分析了在故障率服从指数分布和威布尔分布情况下，HCM 模型和两级增量检查点策略的故障恢复时间开销。从这两个图能够显而易见地看出，HCM 模型具有较少的故障恢复时间开销。

通过对以上三部分时间开销进行叠加可以得到移动云系统总额外时间开销，如图 3.6(a)和图 3.6(b)所示。根据图 3.6(a)中的数据可以看出，在移动云系统的故障概率分布服从指数分布时，HCM 模型相对与两级增量检查点策略而言，虽然设置检查点的时间开销更大，但进行恢复恢复的时间开销减小，从总额外时间开销来看 HCM 模型具有更小时间开销。同样，从图 3.6(b)中可以得出，当移动云系统故障概率分布服从威布尔分布时，采用 HCM 模型的系统中总额外时间开销相对两级增量检查点策略也较小。由此可得出结论，本章所提出的 HCM 模型能有效的为移动云系统提供容错功能，且在一定程度上降低了容错开销。

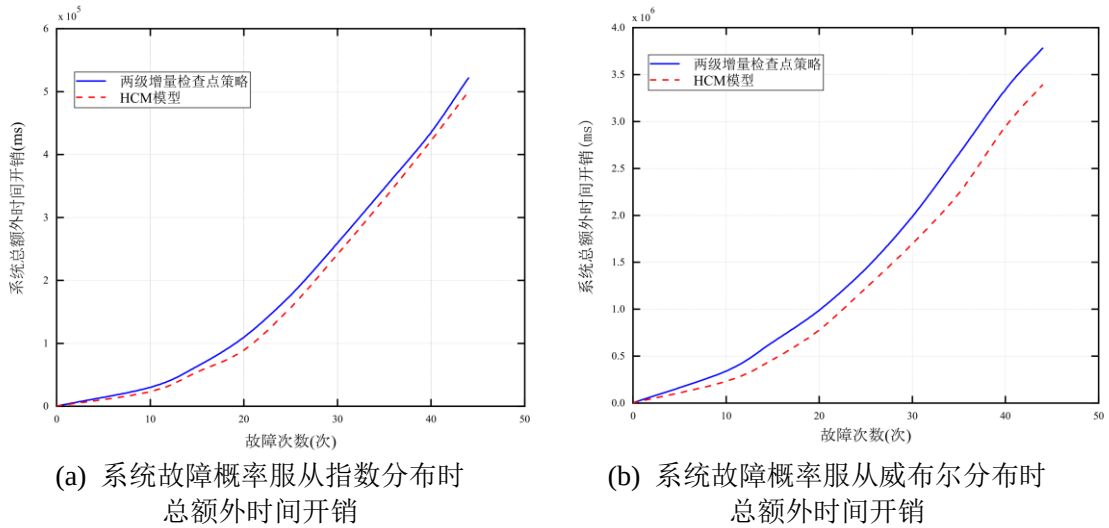


图 3.6 移动云系统总额外时间开销

3.3 本章小结

在本章中，我们提出了基于分级检查点技术的容错模型 HCM。首先，利用随机更新回报理论并结合系统的故障概率分布函数和故障累积分布函数，对移动云系统中故障导致的额外时间开销进行建模。接着，对执行应用程序的总额外时间开销进行最小化处理确定出设置分级检查点的频率函数。最后，本章对 HCM 模型的

性能进行对比仿真实验验证，结果表明所提出的模型对移动云系统的容错开销有所降低，也在一定程度上提高了移动云 QoS。

由章节 3.2.2 中的对比实验结果分析可知，HCM 模型能够有效降低移动云系统发生故障后重算时间开销、故障恢复时间开销及总额外时间开销。除此之外，该模型还能够有效针对任务层级和数据节点层级故障。但该模型中设置分级检查点的时间开销暂未能得到优化。

第4章 基于超图覆盖的分级检查点存储策略

MCC 为用户提供移动云服务, 通过将应用程序中密集型处理任务卸载到云端服务器执行, 使得他们无需占用移动设备资源, 增强了移动设备的功能。通过云计算与无线网络的整合, 移动应用程序拥有了爆炸式涌现的契机。然而, 应用程序往往因图像处理或执行密集计算等各种任务产生大量数据, 这将导致移动设备处理任务的性能减缓。倘若想要在有限的设备资源和电池容量之间权衡出最佳方案, 对移动设备来说是一项重大挑战。为了向用户提供高效且优质的数据服务及构建良好的移动云数据管理环境, 对移动云系统进行容错处理是势在必行。在前文对基于检查点技术的容错方法^[9-49]进行大量研究之后发现, 现有的基于检查点技术的移动云系统容错方法更趋向于对检查点放置频率进行计算或得出更适应于系统不同故障类型的检查点分级方式。但仅依靠优化检查点放置频率来减小容错开销或者更加准确的检查点分级来区别化容错处理并不足以保证大规模长时间运行的移动云系统可靠性, 尤其当移动云系统中多个数据节点由于故障出现瘫痪情况时, 检查点的可用性难以保障。为解决这类极端难题, 对移动云系统中的检查点采用冗余技术备份是一种有效的解决方案。该策略能够快速地将故障数据节点中正在执行的任务切换至其他节点中进行, 将移动云系统损失降到尽可能低的限度。

通过对现有研究成果进行深入分析后发现, 得益于移动云中数据节点的分布式存储系统, 检查点数据冗余存储才能得以技术突破^[58,59]。在传统的存储技术中, 例如 Hadoop 的分布式文件系统(Hadoop Distributed File System, HDFS)默认采用为原始检查点文件创建三个副本, 并将它们按照基于机架感知的随机存储策略^[60]放置于可用数据节点中。这种方法将第一个检查点的副本随机存储在一个可用的数据节点中; 然后, 再筛选出不重复的机架并将第二副本放置于该机架中任意的数据节点中; 最后, 在此机架中筛选出与第二副本放置位置不同的数据节点存储第三副本。依据这种机制, 检查点文件的可用性在一定程度上得到保证。但是这种机制也有其自身的不足之处。最直观的是, 随着移动应用程序处理时间线的推移, 检查点文件的大小也在随之增加, 这为存储这些副本的数据节点带来额外难以负担的存

储和负载压力；其次，该随机放置方法还忽略了移动云中数据节点的地理位置对系统性能和移动云 QoS 的负面影响。

本文在上一章节提出了基于分级检查点技术的移动云容错模型(HCM)，针对不同类型故障提供不同层级检查点进行更加精确的容错。由于诸如数据节点宕机这类永久性故障发生概率极小，而一旦发生将导致数据节点中所有数据和文件均不可用（包括检查点文件）。为了保证检查点的可用性，以确保更高的移动云 QoS，本章主要研究对 Level-2 型检查点进行冗余备份操作及为其各个副本的存储做决策。首先，本章采用超图覆盖到移动云拓扑结构上的方式将各数据节点和联合处理移动应用程序的多个数据节点进行抽象，接着通过基于超图覆盖存储的分级检查点存储策略(Hypergraph Coverage Storage Strategy, HCSS)为分级检查点及副本确定布局方案，最后通过 CloudSim 进行仿真实验验证算法有效性。

4.1 移动云中分级检查点的超图覆盖存储策略

最近为了在给定空间中进行最佳数据分配以减少数据的传输等待时间，图论已被并入 MCC 中。考虑到移动云中移动应用程序的处理需要多个数据节点联合执行，因此，检查点及其副本与存储它们的数据节点之间形成多对多关联。也就是说，检查点的多个副本将存储于不同数据节点中，而每个数据节点上也可以存储不同检查点的副本。在传统的图论中，通过邻接矩阵可以准确的刻画二元关联，但这明显不适用于表示移动云中检查点副本存储位置与数据节点之间的多对多关联。为了解决这个问题，首先依据超图理论中顶点与超边之间多对多关联的思想，对移动云拓扑结构进行超图结构覆盖。为了描述简便，下文将原始检查点及其两个副本统称为检查点副本。再定义关系矩阵表示检查点副本与数据节点之间的关联。最后为三个副本确定存储位置。

4.1.1 超图覆盖方法

在第 2.2.1 章中已经给出超图 H 的数学定义。超图是图的泛化，它其中的超边由任意数量的顶点组成。在本章节中将描述出超图覆盖移动云拓扑结构的方法：

定义 4.1 超图覆盖顶点集 是指将移动云系统中每个数据节点映射到超图的顶点上, 所有数据节点构成一个顶点集, 用 $V = \{v_1, v_2, \dots, v_n\}$ 表示, n 为移动云系统中数据节点的数量。

现设定顶点集 V 的子集 D_i , 子集 D_i 中的元素为联合执行第 i 个移动应用程序的所有数据节点。

定义 4.2 超图覆盖超边集 是指集合 $E = \{E_1, E_2, \dots, E_m\}$, 其中, $E_1, E_2, \dots, E_m$ 分别表示连接移动云系统中子集 $D_1, D_2, \dots, D_m$ 中所有数据节点的各项超边。

因此, 由超图的顶点集和超边集构成的移动云系统超图覆盖结构可以定义为 $H = (V, E)$, 其中 $E_j \neq \emptyset$, ($j = 1, 2, \dots, m$) 且 $\bigcup_{j=1}^m E_j = V$; m 为移动云系统中执行移动应用程序的总数量, $\emptyset$ 为空集。

通过将移动云系统数据节点映射为超图顶点与将系统中处理各特定移动应用程序的数据节点映射为超边, 可完成移动云系统的超图覆盖。再超图覆盖移动云模型构建完成后, 需要一种量化各数据节点之间距离的技术。为此, 本章采用海明距离来对各数据节点之间的距离进行定量分析。由于 n 维无向超图由 $2n$ 个顶点构成。所以需要对超图覆盖后的移动云系统中各数据节点采用 n 位二进制编码的方式进行标记。

例如, 在一个系统规模为十六个数据节点的超图覆盖移动云系统模型中, 需要用四位二进制编码标记各数据节点。则各数据节点的二进制编码如图 4.1 所示。

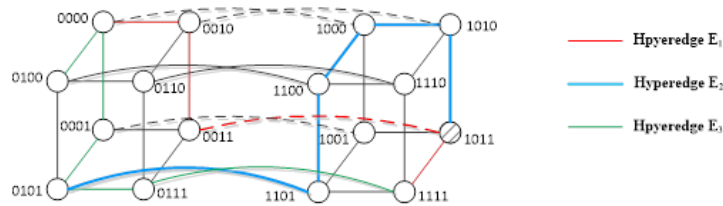


图 4.1 超图覆盖模型示意图

4.1.2 分级检查点存储策略

根据章节 3.1.2 中获取到的移动云系统分级检查点频率函数, 可以得出各层级检查点的时间序列。对于 Level-1 型检查点, 将其存储于内存中。这种做法归因于内存具有更快的存取速度, 并且在该任务成功执行完毕后, Level-1 型检查点即可

释放,不再占用资源。对于 Level-2 型检查点,系统需为其再制作两个检查点副本以保障其在发生数据节点层级的物理性故障时仍具有可用性。

定义 4.3 第一副本存储位置 是指将数据节点中生成的 Level-2 型检查点原始文件定义为第一副本,并将其存储于该数据节点的稳定存储器中,该稳定存储器称为 localHDFS。

定义 4.4 第二副本存储位置 生成 Level-2 型检查点的数据节点至少属于一条超边。将该节点所属的所有超边中的数据节点构成一个超边节点集合EV。然后,在该集合EV中选取一个利用率低数据节点的稳定存储器进行存储,该稳定存储器称为 hyperedgeHDFS。

由于一个数据节点可能属于多条超边,为更加直观准确刻画出检查点副本与数据节点之间多对多关系,提出对数据节点和超边建立关系矩阵。现在假设超图 $H = (V, E)$ 的关系矩阵为 $P(p_{ij})$, 其中关系矩阵 P 中的第 i 列表示超图 H 中的顶点 $i (i = 1, 2, \dots, n)$, 关系矩阵 P 中的第 j 行表示超图 H 中的超边 $j (j = 1, 2, \dots, m)$ 。若关系矩阵 P 中第 i 列所表示的数据节点属于关系矩阵 P 中第 j 行所表示的超边,即 $v_i \in E_j$, 则 $p_{ij} = 1$; 否则 $p_{ij} = 0$ 。因此,数据节点的利用率可表示为:

$$d_H(v_i) = \sum_{j=1}^m p_{ij} \quad (4.1)$$

若存在多个数据节点利用率相同低的情况,则在其中选择数据节点的剩余存储量最高的数据节点进行第二副本存储。

定义 4.5 第三副本存储位置 是指 Level-2 型检查点原始文件另一个副本,需在与该数据节点直连的所有数据节点组成集合EC中,选取一个优先权重最小的数据节点的稳定存储器进行存储,该稳定存储器称为 D-connectHDSF。

考虑到与生成 Level-2 型检查点的数据节点的直连节点属性可能不同,则其节点各类资源使用情况也不尽相同。为此,采用优先权重A来判断适合存储第三副本的数据节点。优先权重计算公式如式(4.2)所示:

$$A = a \cdot N_{mem} + b \cdot N_{cpu} + c \cdot N_{sysload} \quad (4.2)$$

其中, N_{mem} 表示数据节点中剩余存储资源的百分比, N_{cpu} 表示数据节点的 CPU 资源占用百分比, $N_{sysload}$ 表示数据节点平均每五分钟负载, a 表示 N_{mem} 的优先比例系数, b 表示 N_{cpu} 的优先比例系数, c 表示 $N_{sysload}$ 的优先比例系数, a, b, c 由移动云系统管理员根据资源需求情况来自定义设置,且 $a + b + c = 1$ 。

4.2 超图覆盖存储算法

4.2.1 算法描述与性能分析

通过第 3.1 章节所提出的 HCM 模型可获取到分级检查点序列。然后利用本章提出的基于超图覆盖的分级检查点存储策略对 Level-2 型检查点及其副本进行存储位置决策。综合考虑系统中各数据节点利用率情况, 根据 HCSS, 对检查点进行可用性保证。图 4.2 展示了基于超图覆盖存储的分级检查点策略的功能模块图。

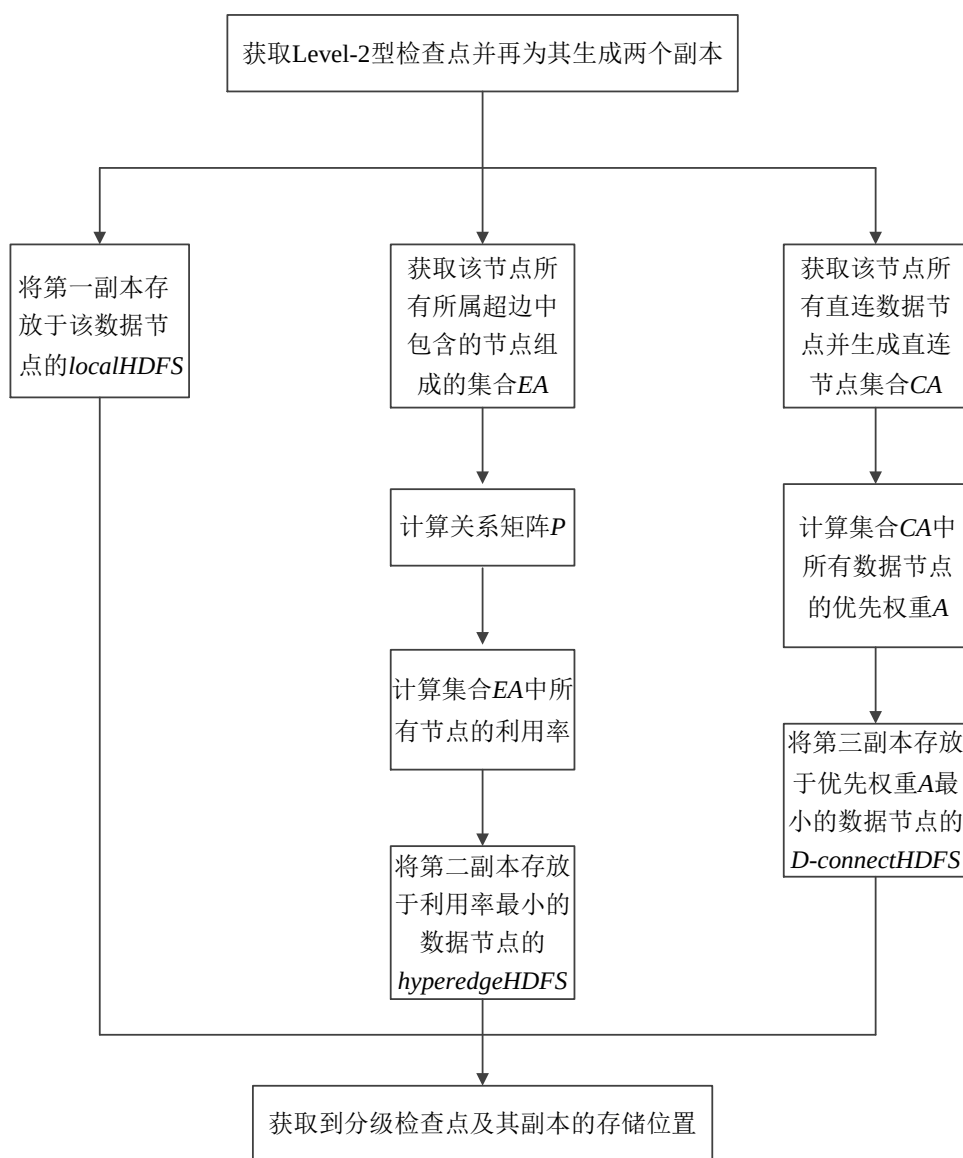


图 4.2 基于超图覆盖的分级检查点存储策略的功能模块图

基于超图覆盖的分级检查点存储算法伪代码如图 4.3 所示：

算法 4.1 HCSS

```

输入 : Level-2 型检查点时间序列  $Level - 2CKPT_i$ 
        系统中正在执行的移动应用程序集合  $Application$ 
输出 : 检查点副本存储位置
1:   获取系统中联合处理移动应用程序  $Application_j$  的数据节点编号, 并生成超边  $E_j$ 
2:   将数据节点映射为超图顶点, 并利用超边  $E_j$  对系统进行超图覆盖
3:   While  $t = Level - 2CKPT_i$  do
4:       获取 Level-2CKPT 及其所属数据节点编号  $Node$ , 并为其生成两个副本
5:       将第一副本存放于  $Node$  的  $localHDFS$  中
6:       For  $Node \in E_j$  do
7:           获取  $Node$  所属所有超边中的数据节点组成的集合  $EV$ 
8:           计算关系矩阵  $P$ 
9:           利用公式(4.2)计算集合  $EV$  中各数据节点的利用率  $d_H(v_i)$ 
10:          获取  $d_H(v_i)_{min}$  对应的数据节点编号
11:          If  $d_H(v_i)_{min}$  对应的数据节点编号的个数  $> 1$  then
12:              获取这些数据节点中剩余存储量最大的数据节点编号  $Node_{mmax}$ 
13:              return  $Node_{mmax}$ 
14:          Else
15:              return  $d_H(v_i)_{min}$  对应的数据节点编号
16:              将第二副本存放于所选数据节点的  $hyperedgeHDFS$  中
17:          End If
18:       End For
19:       For  $Node \in EC$  do
20:           利用公式(4.2)计算集合  $EC$  中所有数据节点的优先权重  $A$ 
21:           return  $A_{min}$  对应的数据节点编号
22:           将第三副本存放于所选数据节点的  $D-connectHDFS$  中
23:       End For
24:   End While

```

图 4.3 基于超图覆盖的分级检查点存储算法伪代码

假设移动云系统中共有 j 个应用程序在并行处理, 则系统中构建这 j 条超边需要的时间复杂度为 $O(jlgj)$ 。当根据算法 3.1 系统共获取到 i 个 Level-2 型检查点时, 集合 EV 中共有 m 个数据节点, 且集合 EC 中共有 k 个数据节点, 因此, 对检查点副本进行存储需要的时间复杂度为 $O(i(1 + mlgm + klgk))$ 。由此可见, 基于超图覆盖的分级检查点存储算法的总时间复杂度为 $O(jlgj + i(1 + mlgm + klgk))$, 其中常数 j 为系统中超边数量, 对检查点副本存储位置不产生决定性影响, 而集合 EV 中数据节点数量 m 和集合 EC 中数据节点数量 k 以及 Level-2 型检查点数量 i 对时间复杂度影响最大。因此, 基于超图覆盖的分级检查点存储算法的时间复杂度可简化为 $O(imlgm + iklgk)$ 。

4.2.2 实验结果及分析

在本小节中,为了验证本论文提出算法的有效性,进行了两个对比实验。首先,在实验一中,将会把本章节提出的基于超图覆盖的分级检查点存储算法运用到数据节点规模为 128 节点和 256 节点的移动云系统中,从而对 Level-2 型检查点及副本进行存储位置决策。并通过与基于机架感知的随机存储策略^[60]进行对比,主要分析检查点存储任务执行时间和系统中各数据节点剩余存储量情况。然后,在实验二中,对 HCSS 算法和谷雷等人^[61]所提出的多级检查点的存储策略进行对比实验验证。最后,进一步深入分析检查点可用性和移动云系统的系统负载情况,用以验证本章提出策略的有效性。

根据 M. Jassas 等人^[62]对 CPU、内存等云资源与故障之间关联度的分析,本文在进行实验时,设置优先比例系数为: $a = 0.5$ 、 $b = 0.3$ 、 $c = 0.2$,系统管理员可以根据自身需求对这三个优先比例系统进行自定义设置。此外,本实验中所采用的检查点的读写模式为一次读写,永久性访问模式。

实验一 (HCSS 与基于机架感知的随机存储策略对比实验):

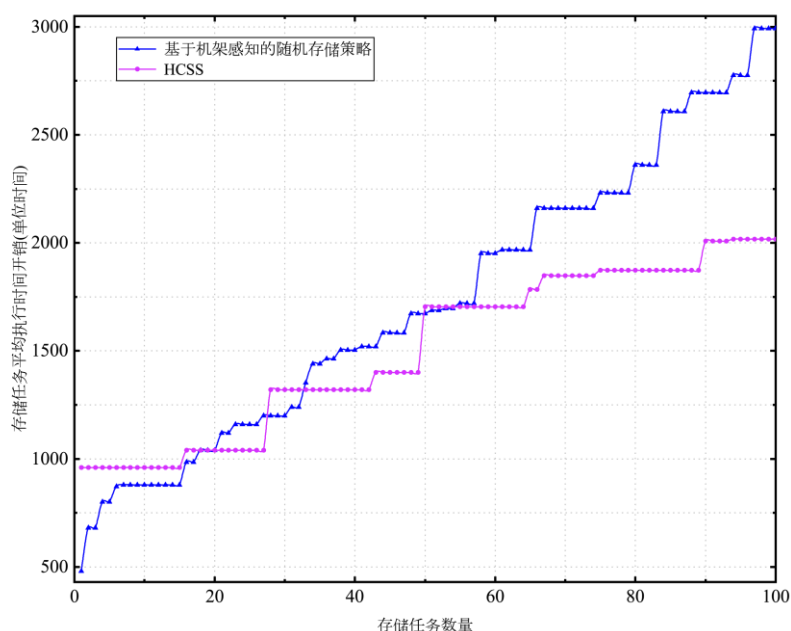


图 4.4 系统规模为 128 节点时存储任务平均执行时间

图 4.4 展示的是系统数据节点规模为 128 节点,系统总存储任务量为 100 个任务,并且移动应用程序的数据集大小为 30GB 时,存储任务平均执行时间变化。即

相同存储任务量情况下,平均执行时间越低,算法执行时间效率越高。从图中可以看到,当系统中总存储任务量小于18个任务时,采用基于机架感知的随机存储策略比采用HCSS的平均执行时间低。经分析可发现,在系统总存储任务量小的情况下,基于机架感知的随机存储策略有优势的原因在于,其算法中判断和处理数据小,检查点副本放置决策更容易执行。然而HCSS需先花时间计算数据节点利用率和数据节点优先权重,再进行检查点副本放置决策。当系统中总存储任务量介于[18,58]时,采用基于机架感知的随机存储策略的系统中平均执行时间一直处于不断增加趋势。而采用HCSS的系统中平均执行时间呈现阶梯形增长趋势,也就是说,在三个陡增点后,存储任务平均执行时间几乎保持平稳。通过获取系统中负载情况和服务器CPU占用详情后发现,出现这三个陡增点是因为该时刻系统中移动应用程序的计算任务量增大,而系统中设置的计算任务优先级比存储任务优先级更高所导致。当系统中总存储任务量介于(58,100]时,采用HCSS的系统存储任务平均执行时间明显比采用基于机架感知的随机存储策略的系统低。

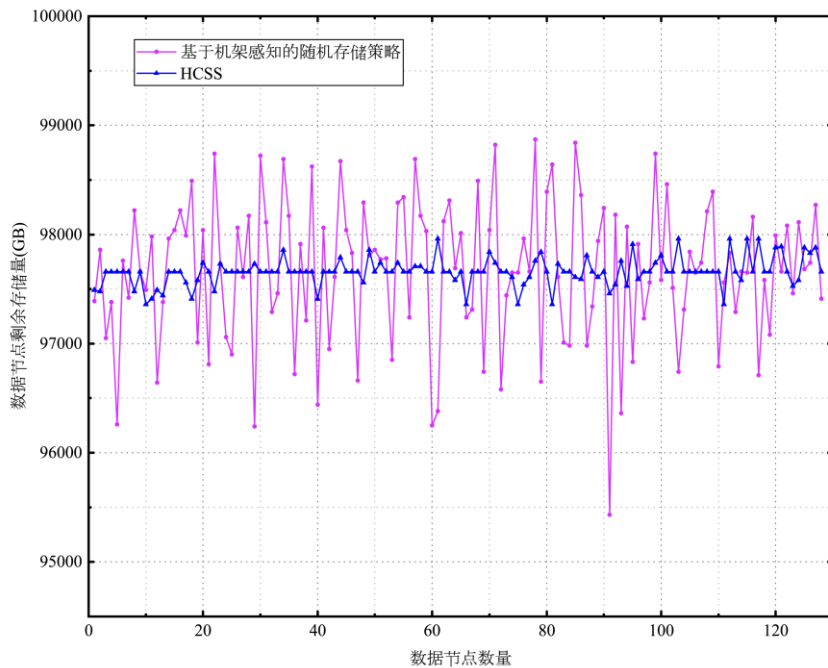


图 4.5 系统规模为 128 节点时各数据节点剩余存储量

图 4.5 为系统中各数据节点的剩余存储量情况。由于在确定时刻数据节点中剩余存储量高,既反映出该数据节点当前的利用率低,也反映出此刻该数据节点中存储检查点所用存储量低。换言之即是,基于机架感知的随机存储策略是根据机架信

息和机架中数据节点信息随机存储检查点副本，数据节点剩余存储量低则说明其中存储的检查点副本多，数据节点的利用率高，该数据节点的系统负载相对较高。但 HCSS 则是综合考虑数据节点利用率和根据优先比例系数进行存储位置选择，剩余存储量高的数据节点，其节点利用率低，系统负载也相对较低。因此，通过图 4.5 中数据显示出，蓝色曲线上下波动幅度小，即采用 HCSS 进行检查点副本存储位置决策的系统，数据节点的剩余存储量相对平均。然而，红色曲线上下波动幅度大，说明采用基于机架感知的随机存储策略决定检查点副本存储位置的系统，各数据节点的剩余存储量差异很大。

现将移动云系统的数据节点规模增大到 256 个数据节点，系统总存储任务数量增大到将近 1500 个任务，并且移动应用程序的数据集为 50GB。图 4.6 展示了存储任务的平均执行时间。采用 HCSS 的系统中存储任务的平均执行时间相对与基于机架感知的随机存储策略减少了 33 ~ 40%。图 4.7 展示了移动云系统中各数据节点剩余存储量的对比情况。从图中数据可以明显看出，采用 HCSS 决策出检查点副本存储位置的系统中各数据节点剩余存储量差异较小，而采用基于机架感知的随机存储策略选取的检查点副本存储位置的系统中，各数据节点剩余存储量差异较大。

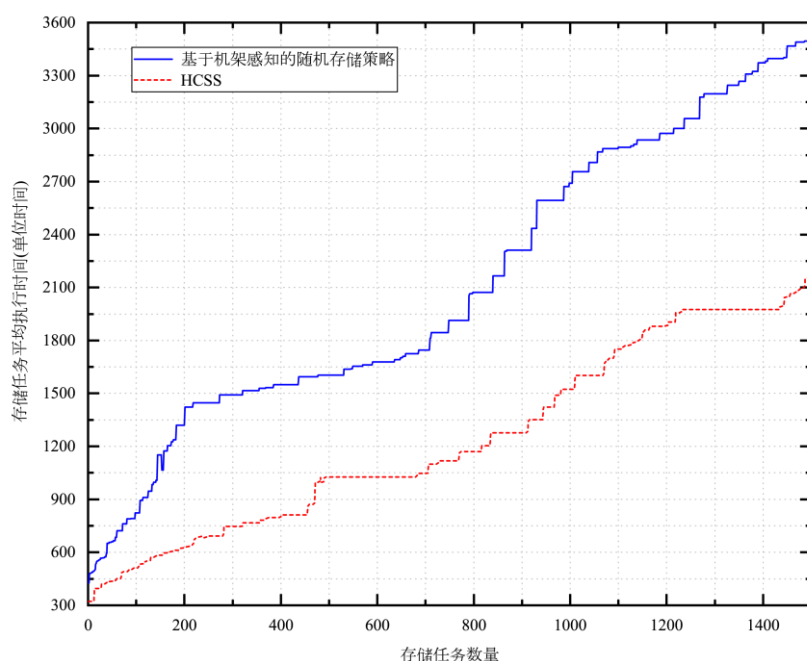


图 4.6 系统规模为 256 节点时存储任务平均执行时间

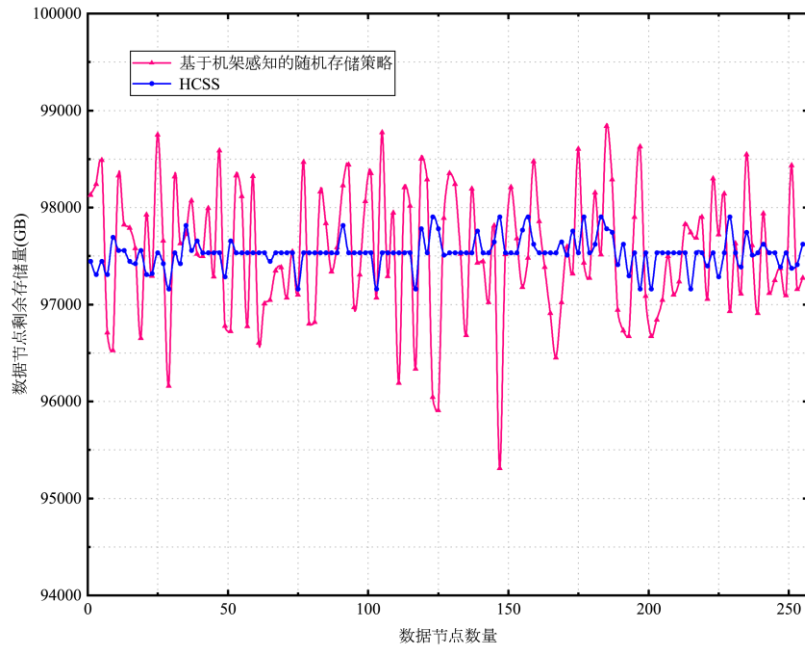


图 4.7 系统规模为 256 节点时各数据节点剩余存储量

综合图 4.4、图 4.5、图 4.6 和图 4.7 中展示的实验结果经深入分析可得出，在系统规模和存储任务量较大的场景中，HCSS 具有较好的性能。由于存储任务的平均执行时间较低，各数据节点的剩余存储量差异较小，因此，采用 HCSS 的系统拥有更加均衡的负载。但 HCSS 算法的时间复杂度相对基于机架感知的随机存储策略而言更高，因此在系统规模和存储任务量较小的场景中，基于机架感知的随机存储策略能够具有一些略微的优势。

实验二（HCSS 与多级检查点存储策略）：多级检查点存储策略^[61]由磁盘检查点和双内存检查点构成。其中，磁盘检查点存储在生成节点的稳定存储器中；内存检查点将在生成节点的内存中存储，再生成一份检查点文件的备份保存于集群中其他某节点的内存中。鉴于此，本实验设定系统数据节点规模为 32 节点，并取 15 个故障周期。分别对采用 HCSS 的系统和采用多级检查点存储策略的系统中，检查点存储任务执行时间和系统中各数据节点剩余存储量情况进行对比。

如图 4.8 所展示的是系统中总存储任务量为 60 时，HCSS 和多级检查点存储策略下存储任务平均执行时间。图中可以明显的看出，红色点线所代表的采用 HCSS 系统中存储任务平均执行时间比绿色线所代表的采用多级检查点策略系统中存储任务平均执行时间少 20% ~ 35%。

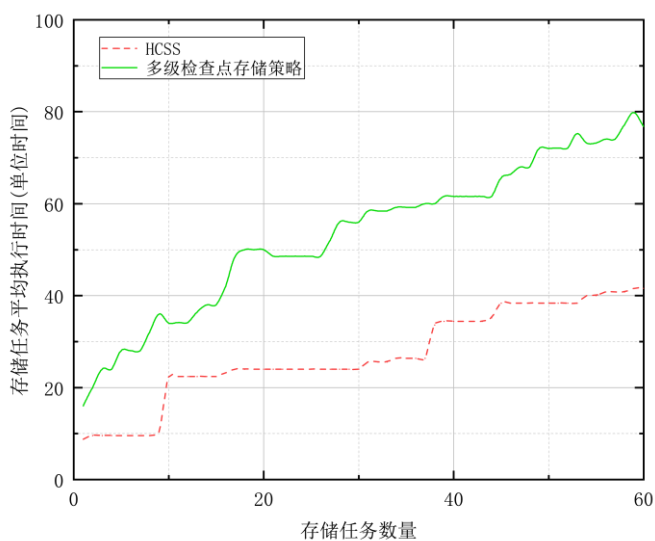


图 4.8 系统规模为 32 节点时存储任务平均执行时间

如图 4.9 所示, 采用 HCSS 的系统中各数据节点剩余存储量比采用多级检查点存储策略的系统中各节点剩余存储量差异更小。通过将各数据节点剩余存储量取平均值后得知, HCSS 策略下各节点平均剩余量比多级检查点存储策略下各节点平均剩余量多 14%。说明 HCSS 策略中检查点冗余备份的在存储资源占用方面优于多级检查点存储策略, 且 HCSS 中各检查点副本的存储位置与各数据节点的剩余存储量和系统负载相关联, 也就是说, HCSS 在均衡系统中各节点负载方面也是有效的。

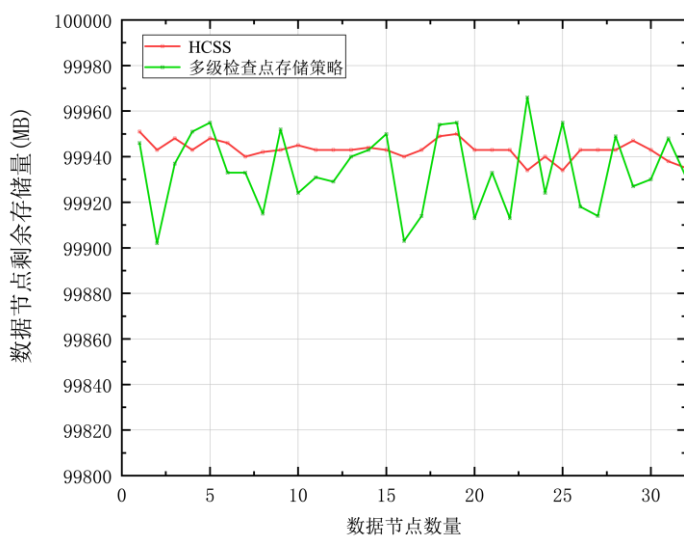


图 4.9 系统规模为 32 节点时各数据节点剩余存储量

4.3 本章小结

本章在第三章研究了基于分级检查点技术的容错模型(HCM)基础上,继续讨论了影响移动云系统容错功能的另一因素,分级检查点及副本的存储位置。首先将移动云系统拓扑结构进行超图覆盖,将系统中数据节点映射为超图的顶点,将系统中处理每个应用程序的数据节点集合映射为一条超边。接着,提出了基于超图覆盖的分级检查点存储策略(HCSS)对分级检查点及副本进行存储位置决策。最后,对该策略进行了仿真实验验证其有效性。实验结果数据显示,相对比于分布式存储系统中默认的基于机架感知的随机存储策略而言,HCSS 在存储任务执行时间和均衡节点负载方面具有明显优化效果。利用保证分级检查点可用性的方式有效地提高了移动云系统的 QoS。

第5章 分级检查点容错系统的设计与实现

本文第三章和第四章节中，主要介绍了基于分级检查点技术的容错模型和基于超图覆盖的分级检查点存储策略。本章将具体描述融合以上模型和策略的分级检查点容错系统的设计与实现。

5.1 系统设计

5.1.1 功能设计

得益于海量数据分析技术的不断创新发展和信息可视化技术取得的卓越进步，目前，大多数移动云系统已经开始利用图表、动态效果等交互式视觉表现形式将抽象数据直观地展示给用户。这项技术增强了移动云系统中数据的呈现效果，方便用户更加明确数据变化走势，还有助于进一步深入分析数据变化中包含的隐藏信息。因此，本文基于 Python Django 框架开发了一个 Web 系统，该 Web 系统能够为移动云系统管理员展示系统中的分级检查点信息及在超图化覆盖的移动云拓扑中分级检查点存储位置。主要具有以下功能：

1. 系统管理员登录：移动云系统管理员登录后，可以观察到系统中设置分级检查点的信息、分级检查点存储信息、还可以完成系统参数调整。
2. 分级检查点信息展示：将移动云系统中设置的分级检查点及其开销等数据采用图表进行展示，便于系统管理员准确掌握系统中容错开销情况，也方便系统管理员进一步分析系统故障率情况，衡量系统可靠性。
3. 分级检查点存储展示：以列表的形式展示检查点及其副本的存储位置；同时还为系统管理员展示系统中各数据节点上的剩余存储量。
4. 系统参数详情展示：以列表的方式显示出系统中各项可配置参数，允许管理员用户对相关参数数值做出调整。

5.1.2 模块设计

分级检查点容错系统开发环境以 PyCharm 为编译工具，采用由 Python 语言控制的基于 MTV 模式的 Django 框架，最终通过 WebUI 页面与系统管理员进行系统信息交互。根据系统功能的需求分析，主要在系统中实现四个模块。如图 5.1 所示的是本文开发原型系统的结构图。系统管理员登录系统后可以在网站首页进行信息浏览和操作，系统模块主要包括系统管理员登录模块、分级检查点信息展示模块、分级检查点存储展示模块以及系统参数详情模块：

1. 系统管理员登录模块：该模块主要完成验证管理员账号密码匹配，匹配成功则登录进入分级检查点容错系统主页，匹配失败则要求重新登录；以及完成管理员用户注册。

2. 分级检查点信息展示模块：该模块完成的功能是按照第三章所提出的分级检查点频率函数推算出的时间序列用图表的方式记录分级检查点设置的时刻，并展示分级检查点开销。让系统管理员清晰地掌握分级检查点设置时刻及系统容错开销，以便对系统进行深入分析，为系统管理员调整系统参数提供相关依据。

3. 分级检查点存储展示模块：该模块将利用列表展示数据节点中存储检查点及副本的信息，同时展示存储任务执行时间以及总存储任务量。

4. 系统参数详情模块：该模块以列表的形式展示出系统中数据节点的优先比例系数详细信息。

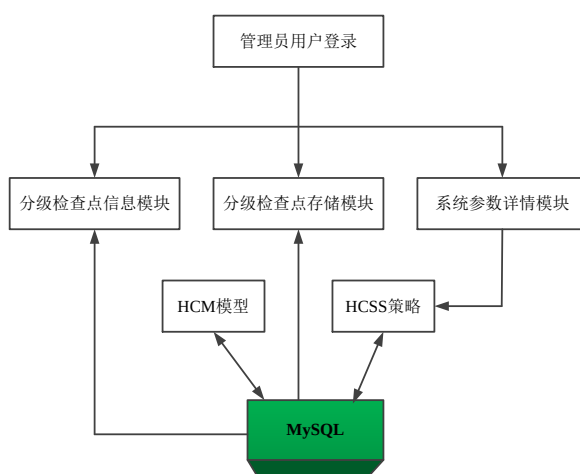


图 5.1 分级检查点容错系统结构图

5.1.3 数据库设计

本章所述分级检查点容错系统中使用 MySQL5.7 版本数据库，主要的数据表结构设计如下表 5.1-5.4：

1. 管理员用户信息表(UserInfo)

表 5.1 管理员用户信息表(UserInfo)		
字段名	类型	字段描述
autoID	int	用户唯一标识符
userID	int	用户名
pwd	varchar	用户密码

2. 数据节点信息表(DateNodeInfo)

表 5.2 数据节点信息表(DataNodeInfo)		
字段名	类型	字段描述
autoID	int	数据节点唯一标识符
DataNodeID	varchar	存储检查点的数据节点编码
taskID	int	移动任务唯一标识符
Failure	int	数据节点上发生故障的数量
totalTime	double	任务总执行时间
taskProcess	double	任务执行进度

3. 分级检查点信息表(HCheckpointInfo)

表 5.3 分级检查点信息表(HCheckpointInfo)		
字段名	类型	字段描述
autoID	int	分级检查点唯一标识符
taskID	int	移动任务唯一标识符
ckptType	char	检查点层级唯一标识符
ckptTime	double	分级检查点时刻
settingTime	double	分级检查点开销

4. 分级检查点存储信息表(CkptStorage)

表 5.4 分级检查点存储信息表(CkptStorage)

字段名	类型	字段描述
autoID	int	分级检查点唯一标识符
taskID	int	移动任务唯一标识符
ckptType	char	检查点层级唯一标识符
sTaskID	int	存储任务唯一标识符
ckptTime	double	分级检查点时刻
PDataNodeID	int	生成检查点的数据节点编码
DataNodeID	varchar	存储检查点的数据节点编码
sTaskTime	double	存储任务执行时间

以上数据表中，管理员用户信息表(UserInfo)用于存储移动云系统管理员信息，主要用于登录模块验证登录信息是否匹配；分级检查点信息表(HCheckpointInfo)用于存放移动云系统中设置分级检查点的详细信息，包括检查点序号、其对应的检查点层级信息、其对应的设置检查点时刻的系统时间、设置检查点所产生的系统时间；分级检查点存储信息表(CkptStorage)用于存储检查点生成数据节点信息以及其副本存放数据节点信息，其中 DataNodeID 字段使用 JSON 格式存储检查点副本的存储数据节点信息。

5.2 系统实现与界面展示

本文提出的分级检查点容错系统通过浏览器访问 Web 页面展示出来。该系统中刷新时间设置为 30 秒进行一次刷新，用户除了可以刷新数据以外，还可以对系统参数进行自定义调整。分级检查点容错系统主要展示 4 个用户界面：

如图 5.2 所示的是移动云系统管理员登录的界面。当用户输入的账号密码通过认证后，即可成功登录。若暂无可用账号，可先进行注册，如图 5.3 所示页面。再使用注册的账号登录并访问分级检查点容错系统。



图 5.2 管理员用户登录界面

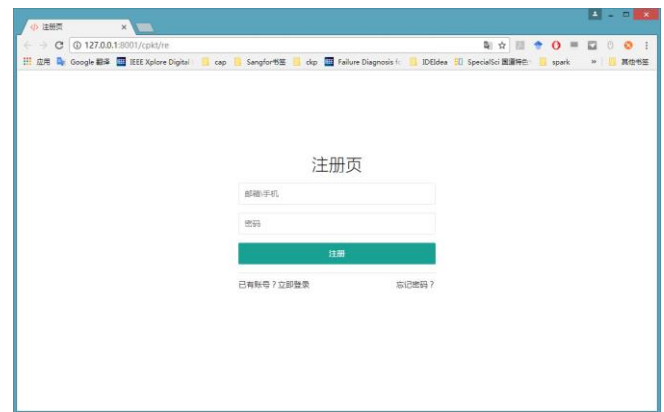


图 5.3 管理员用户注册界面

图 5.4 展示出了分级检查点管理页面，在该界面可以看到分级检查点系统各方面统计的数据信息。包括系统规模、各层级检查点已设置数量、系统故障数量、任务总执行时间、分级检查点成本开销、分级检查点设置详情。柱状图显示的是当前系统中设置各个检查点所用的时间开销；分级检查点设置详情列表中可以清楚了解各个检查点的类型和设置时刻；还可以通过日期选择的方式，查看具体日期系统所设置的检查点信息。

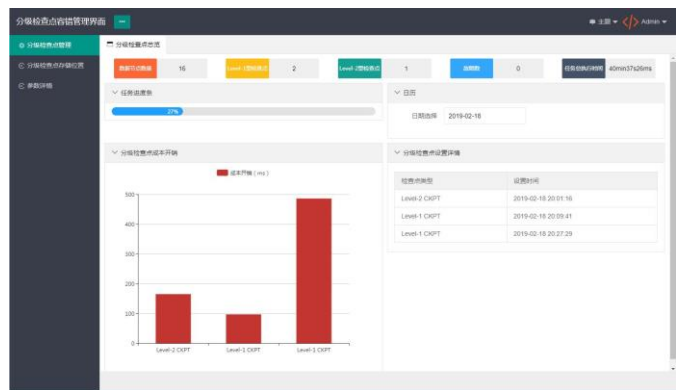


图 5.4 分级检查点管理界面

图 5.5 为分级检查点存储位置展示界面。该界面能够使管理员用户直观地看出当前系统中数据节点数量、存储任务数量及存储任务总耗时。还对数据节点中的检查点存储数量进行列表展示。通过图 5.5 我们可以非常明确地了解存储策略的有效性。

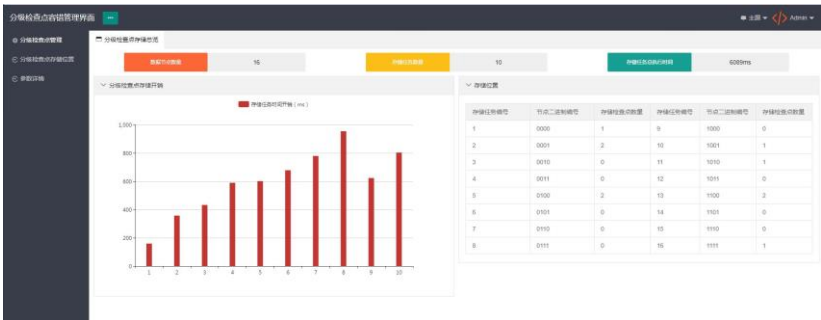


图 5.5 分级检查点存储位置界面

图 5.6 为分级检查点系统中各类参数设置情况，包括允许管理员用户进行手动配置的优先比例系数 a 、 b 、 c ；以及通过数据捕获所得到的各数据节点编号及数据节点上 CPU、MEM、平均每五分钟负载、剩余存储量百分比等信息。

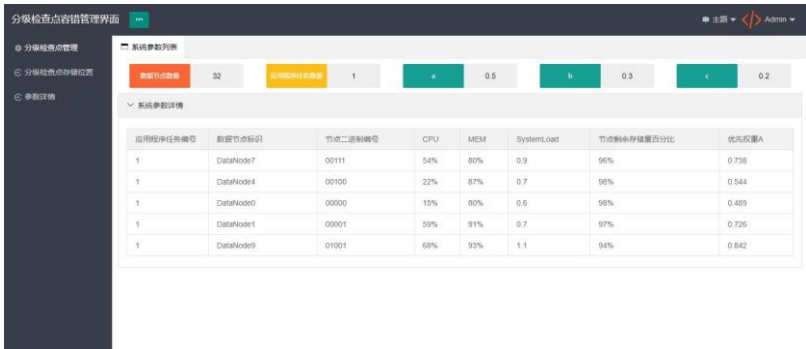


图 5.6 系统参数列表详情界面

5.3 本章小结

本章提出分级检查点容错系统的思想，设计并且基于 Django 框架实现了该原型系统。首先，本章进行了系统设计，明确系统需要实现四大功能模块包括：系统管理员登录模块，分级检查点信息展示模块，分级检查点存储展示模块以及系统参数详情模块。然后，对四大模块进行具体的功能项设计，同时还设计了数据库用于

存储系统数据。接着，实现了该容错系统的原型。最后，对分级检查点容错系统实现代码进行单元测试，对系统进行功能测试和集成测试，并展示了部分测试结果。

第6章 总结与展望

6.1 总结

在云计算和移动设备迅速发展的助推下，MCC 快速崛起，移动云服务也逐渐融入各个领域。移动云的规模、服务使用量、网络结构和应用场景之间相辅相成，其在不断为人们生活便利的同时，也面临着巨大的可靠性挑战：系统规模大且网络环境复杂导致的故障定位难；应用程序种类繁多导致故障产生的损失难以明确地量化等等。从国防安全领域到企业生产、从市政统筹规划到人们生活的各个方面，倘若移动云出现不可规避的故障，不仅仅是给用户和服务提供商造成巨额经济损失，情况严重时甚至引发灾难性事件。故此，移动云的容错成为亟待解决的问题，基于分级检查点技术的容错手段成为了企业和学术界广泛研究的热门话题。

使用分级检查点技术对移动云资源进行容错时，由于故障事件随机发生，难以预测其确切出现时间。因此，分级检查点的设置频率直接影响到系统容错性能和开销。目前，针对容错开销问题已有一些研究成果。但针对移动云环境下不同故障类型的研究尚少。此外，针对检查点的可用性保证的研究也大多基于分布式文件系统默认策略。围绕如何解决上述问题，本文主要贡献如下：

1. 针对移动云系统容错开销问题，本文提出基于分级检查点技术的容错模型(HCM)，目标在于提高移动云系统可靠性的同时，减少系统容错开销。HCM 模型首先对移动云系统中利用分级检查点技术进行容错所造成的额外总开销进行建模。接着，根据随机更新回报理论并结合系统故障分布函数和累积分布函数对系统总额外开销进行最小化处理并获取分级检查点频率函数，根据该频率函数动态确定设置分级检查点的时刻。最后通过与两级增量检查点策略进行对比实验，验证 HCM 模型的正确性和有效性。

2. 针对检查点可用性和可靠性问题，以超图数学作为理论基础，由简入繁，对分级检查点及副本的存储位置进行决策。本文提出基于超图覆盖的分级检查点存储策略(HCSS)。通过对移动云系统拓扑结构进行超图化建模，提出将移动云系统中的数据节点映射为超图顶点，将联合处理同一个应用程序的数据节点抽象为超图的超边。再对检查点及其副本的存储位置进行动态决策。最后将 HCSS 策略

与基于机架感知的随机存储策略进行实验对比分析,发现在一定程度上,本文提出的基于超图覆盖的分级检查点存储策略计算效率更优,对均衡系统负载有一定优势。

3. 借助于可视化技术的概念和思想,本文基于 Python Django 框架设计并实现了分级检查点容错系统。通过图表的形式动态展示出分级检查点时间序列及检查点存储的数据节点位置,直观明了地为移动云系统管理员展示系统容错数据。

6.2 未来的工作

在已经取得的研究成果基础之上,接下来需要指出的是本研究目前还存在的亟需优化的问题:

1. 发生故障后重新计算系数的取值处理不够精确。虽然在 HCM 模型对发生故障后的重新计算开销进行了计算,但是重新计算系数是根据均值定理进行取值的,在复杂的移动云系统中重新计算系数应针对系统实时情况做动态处理。鉴于此, HCM 模型还需要进一步进行优化。通过提高重新计算系数的精度来进一步提升系统可靠性和降低容错开销。

2. HCSS 算法是针对分级检查点存储位置进行动态决策的策略,本文仅仅考虑了为原始检查点文件制作两个副本的情况。其实在复杂的移动云环境下,为减小系统中数据节点的各项指标压力,检查点文件的副本数量需要动态确定。未来的工作中需要针对检查点副本数量的决策问题进行更深入的研究。

3. 本文设计的分级检查点容错系统单单只是实现了基本的原型。该原型系统虽对需求的功能进行了准确实现,但页面美化、动态刷新、以及内存空间优化方面还有较大的改进空间。未来工作可以考虑将其与实际项目进行结合,并优化上述问题。

参考文献

- [1] Prasad A, Mohole G P, Deore K, et al. Google Cloud Storage using tilt gesture[C]// IEEE International Conference on Advances in Electronics. Pune: IEEE Press, 2016: 338-342.
- [2] Khan A U R, Othman M, Madani S A, et al. A Survey of Mobile Cloud Computing Application Models[J]. IEEE Communications Surveys & Tutorials, 2014, 16(1): 393-413.
- [3] Abolfazli S, Sanaei Z, Ahmed E, et al. Cloud-Based Augmentation for Mobile Devices: Motivation, Taxonomies, and Open Challenges[J]. IEEE Communications Surveys & Tutorials, 2014, 16(1): 337-368.
- [4] Liu Fangming, Shu Peng, Jin Hai, et al. Gearing resource-poor mobile devices with powerful clouds: architectures, challenges, and applications[J]. IEEE Wireless Communications, 2013, 20(3): 14-22.
- [5] Dinh H T, Lee C, Niyato D, et al. A survey of mobile cloud computing: architecture, applications, and approaches[J]. Wireless communications and mobile computing, 2013, 13(18): 1587-1611.
- [6] Chen X. Decentralized computation offloading game for mobile cloud computing[J]. IEEE Transactions on Parallel and Distributed Systems, 2015, 26(4): 974-983.
- [7] Deng S, Huang L, Taheri J, et al. Computation offloading for service workflow in mobile cloud computing[J]. IEEE Transactions on Parallel and Distributed Systems, 2015, 26(12): 3317-3329.
- [8] Singla A, Chandrasekaran B, Godfrey P, et al. The internet at the speed of light[C]// Proceedings of the 13th ACM Workshop on Hot Topics in Networks. New York: ACM Press, 2014: 1-7.
- [9] Young J W. A first-order approximation to the optimum checkpoint interval[J]. Communications of the ACM, 1974, 17(9): 530-531.
- [10] 徐振朋, 门朝光, 李香. 日志检查点回卷恢复策略的检查点周期求解模型[J]. 高技术通讯, 2011, 21(6): 575-580.
- [11] Lim S H, Lee S W, Lee B H, et al. Power-aware optimal checkpoint intervals for mobile consumer devices[J]. IEEE Transactions on Consumer Electronics, 2011, 57(4): 1637-1645.

- [12] Park J S, Chung K S, Lee E Y, et al. Monitoring service using Markov chain model in mobile grid environment[C]// *Advances in Grid and Pervasive Computing*. Hualien: Springer Press, 2010: 193-203.
- [13] Dimitriou I. A mixed priority retrial queue with negative arrivals, unreliable server and multiple vacations[J]. *Applied Mathematical Modelling*, 2013, 37(3): 1295-1309.
- [14] Dimitriou I. A retrial queue for modeling fault-tolerant systems with checkpointing and rollback recovery[J]. *Computers & Industrial Engineering*, 2015, 79(1): 156-167.
- [15] Zhao Juzi, Xiang Yu, Lan Tian, et al. Elastic Reliability Optimization Through Peer-to-Peer Checkpointing in Cloud Computing[J]. *IEEE Transactions on Parallel and Distributed Systems*, 2017, 28(2): 491-502.
- [16] Zhou Ao, Wang Shangguang, Zheng Zibin, et al. On Cloud Service Reliability Enhancement with Optimal Resource Usage[J]. *IEEE Transactions on Cloud Computing*, 2014, 4(4): 452-466.
- [17] Tiwari D, Gupta S, Vazhkudai S S. Lazy checkpointing: exploiting temporal locality in failures to mitigate checkpointing overheads on extreme-scale systems[C]// *Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. Atlanta: IEEE Press, 2014: 25-36.
- [18] Íñigo Goiri, Ferran Julià, Guitart J, et al. Checkpoint-based fault-tolerant infrastructure for virtualized service providers[C]// *2010 IEEE Network Operations and Management Symposium*. Osaka: IEEE Press, 2010: 455-462.
- [19] 黄友富. 面向云平台的协同卷回恢复关键技术研究[D]. 哈尔滨工业大学, 2014.
- [20] Cully B, Lefebvre G, Meyer D, et al. Remus: High availability via asynchronous virtual machine replication[C]// *Proceedings of the 5th USENIX Symposium on Networked Systems Design and Implementation*. Berkeley: USENIX Association, 2008: 161-174.
- [21] Kangarlou A, Eugster P, Xu D. VNsnap: Taking snapshots of virtual networked environments with minimal downtime[C]// *IEEE/IFIP International Conference on Dependable Systems & Networks*. Lisbon: IEEE Press, 2009: 524-533.
- [22] Wang L, Kalbarczyk Z, Iyer R K, et al. VM- μ Checkpoint: Design, Modeling, and Assessment of Lightweight In-Memory VM Checkpointing[J]. *IEEE Transactions on Dependable and Secure Computing*, 2015, 12(2): 243-255.

- [23] Bosilca G, Bouteiller A, Brunet E, et al. Unified model for assessing checkpointing protocols at extreme-scale[J]. *Concurrency and Computation: Practice and Experience*, 2014, 26(17): 2772-2791.
- [24] Plank J S, Li K, Puening M A. Diskless checkpointing[J]. *IEEE Transactions on Parallel and Distributed Systems*, 1998, 9(10): 972-986.
- [25] Egwuotuoha I P, Levy D, Selic B, et al. A survey of fault tolerance mechanisms and checkpoint/restart implementations for high performance computing systems[J]. *Journal of Supercomputing*, 2013, 65(3): 1302-1326.
- [26] Guermouche A, Ropars T, Brunet E, et al. Uncoordinated Checkpointing Without Domino Effect for Send-Deterministic MPI Applications[C]// *IEEE International Parallel & Distributed Processing Symposium*. Anchorage: IEEE Press, 2011: 989-1000.
- [27] Garcia I C, Vieira G M D, Buzato L E. A Rollback in the History of Communication-Induced Checkpointing[J]. *Distributed, Parallel, and Cluster Computing*, 2017,1(1): 1-10.
- [28] Güler Berkin, Özkasap Öznur. Efficient checkpointing mechanisms for primary-backup replication on the cloud[J]. *Concurrency and Computation: Practice and Experience*, 2018, 30(21):1-15.
- [29] Vaidya N H. A Case for Two-Level Recovery Schemes[J]. *IEEE Transactions on Computers*, 1998, 47(6):656-666.
- [30] Di S, Robert Y, Frédéric Vivien, et al. Toward an Optimal Online Checkpoint Solution under a Two-Level HPC Checkpoint Model[J]. *IEEE Transactions on Parallel & Distributed Systems*, 2016, 28(1):244-259.
- [31] Debanjan D, David A, Constantinos E, et al. An Asynchronous Two-Level Checkpointing Method to Solve Adjoint Problems on Hierarchical Memory Spaces[J]. *Computing in Science & Engineering*, 2018, 20(4): 39-55.
- [32] Schanen M, Marin O, Zhang Hong, et al. Asynchronous Two-level Checkpointing Scheme for Large-scale Adjoint in the Spectral-Element Solver Nek5000[J]. *Procedia Computer Science*, 2016, 80: 1147-1158.
- [33] Paun M, Naksinehaboon N, Nassar R, et al. Incremental Checkpoint Schemes For Weibull Failure Distribution[J]. *International Journal of Foundations of Computer Science*, 2014, 21(03): 329-344.

-
- [34] Li Huixian, Pang Liaojun, Wang Zhangquan. Two-Level Incremental Checkpoint Recovery Scheme for Reducing System Total Overheads[J]. PLOS ONE, 2014, 9(8): 1-17.
- [35] Chang HsungPin, Liu YuChieh, Yang ShangSheng. Surviving sensor node failures by MMU-less incremental checkpointing[J]. Journal of Systems and Software, 2014, 87: 74-86.
- [36] Ferreira K B, Riesen R, Bridges P, et al. Accelerating incremental checkpointing for extreme-scale computing[J]. Future Generation Computer Systems, 2014, 30: 66-77.
- [37] Fohry C, Posner J, Reitz L. A Selective and Incremental Backup Scheme for Task Pools[C]// International Conference on High Performance Computing Simulation. Orleans: IEEE Press, 2018: 621-628.
- [38] Mirhosseini A, Agrawal A, Torrellas J. Survive: Pointer-based In-DRAM Incremental Checkpointing for Low-Cost Data Persistence and Rollback-Recovery[J]. IEEE Computer Architecture Letters, 2017, 16(2): 153-157.
- [39] Takizawa H, Amrizal M A, Komatsu K, et al. An Application-Level Incremental Checkpointing Mechanism with Automatic Parameter Tuning[C]// 2017 Fifth International Symposium on Computing and Networking. Aomori: IEEE Press, 2017: 389-394.
- [40] Gomez L A B, Maruyama N, Cappello F, et al. Distributed Diskless Checkpoint for Large Scale Systems[C]// 2010 10th IEEE/ACM International Conference on Cluster, Cloud and Grid Computing. Melbourne: IEEE Press, 2010: 63-72.
- [41] Moody A, Bronevetsky G, Mohror K, et al. Design, Modeling, and Evaluation of a Scalable Multi-level Checkpointing System[C]// Proceedings of the 2010 ACM/IEEE International Conference for High Performance Computing. New Orleans: IEEE Press, 2010: 1-11.
- [42] Gomez L B, Nukada A, Maruyama N, et al. Low-overhead Diskless Checkpoint for Hybrid Computing Systems[C]// 2010 International Conference on High Performance Computing. Dona Paula: IEEE Press, 2011: 1-10.
- [43] Bautista-Gomez L, Tsuboi S, Komatitsch D, et al. FTI: High Performance Fault Tolerance Interface for Hybrid Systems[C]// Proceedings of 2011 International Conference for High Performance Computing. Seattle: IEEE Press, 2011: 1-12.
- [44] Di S, Bouguerra M S, Bautista-Gomez L, et al. Optimization of Multi-level Checkpoint Model for Large Scale HPC Applications[C]// IEEE International

- Parallel and Distributed Processing Symposium. Phoenix: IEEE Press, 2014: 1181-1190.
- [45] Benoit A, Cavelan A, Le Fevre V, et al. Towards Optimal Multi-Level Checkpointing[J]. IEEE Transactions on Computers, 2017, 66(7): 1212-1226.
- [46] Qiang Weizhong, Jiang Changqing, Ran Longbo, et al. CDMCR: Multi-level Fault-tolerant System for Distributed Applications in Cloud[J]. Security and Communication Networks, 2016, 9(15):2766-2778.
- [47] Amrizal M A, Takizawa H. Optimizing Energy Consumption on HPC Systems with a Multi-Level Checkpointing Mechanism[C]// 2017 International Conference on Networking. Shenzhen: IEEE Press, 2017: 1-9.
- [48] Sigdel P, Tzeng N F. Coalescing and Deduplicating Incremental Checkpoint Files for Restore-Express Multi-Level Checkpointing[J]. IEEE Transactions on Parallel and Distributed Systems, 2018, 29(12): 2713-2727.
- [49] Benoit A, Cavelan A, Robert Y, et al. Multi-level Checkpointing and Silent Error Detection for Linear Workflows[J]. Journal of Computational Science, 2018, 28: 398-415.
- [50] Benoit A, Cavelan A, Robert Y, et al. Two-level Checkpointing and Verifications for Linear Task Graphs[C]// 2016 IEEE International Parallel and Distributed Processing Symposium Workshops. Chicago: IEEE Press, 2016: 1239-1248.
- [51] Jangjaimon I, Tzeng N F. Adaptive Incremental Checkpointing via Delta Compression for Networked Multicore Systems[C]// 2013 IEEE 27th International Symposium on Parallel and Distributed Processing. Boston: IEEE Press, 2013: 7-18.
- [52] P. Erdős, Spencer J. Imbalances in k-colorations[J]. Networks, 2010, 1(4): 379-385.
- [53] 王建方. 超图的理论基础[M]. 高等教育出版社, 2006: 2-49.
- [54] Zheng Gengbin, Ni Xiang, Laxmikant V K. A Scalable Double in-memory Checkpoint and Restart Scheme Towards Exascale[C]// IEEE/IFIP International Conference on Dependable Systems and Networks Workshops. Boston: IEEE Press, 2012: 1-6.
- [55] Ross S M. 随机过程[M]. 龚光鲁, 译. 机械工业出版社, 2013: 89-92.
- [56] Naksinehaboon N, Liu Yudan, Leangsuksun C, et al. Reliability-Aware Approach: An Incremental Checkpoint/Restart Model in HPC Environments[C]// 2008 Eighth IEEE International Symposium on Cluster Computing and the Grid. Lyon: IEEE Press, 2008: 783-788.

-
- [57] Yan Ying, Gao Yanjie, Chen Yang, et al. Tr-spark: Transient computing for big data analytics[C]// Proceedings of the Seventh ACM Symposium on Cloud Computing. Santa Clara: ACM Press, 2016: 484-496.
- [58] Li Wenhao, Yang Yun, Yuan Dong. A Novel Cost-Effective Dynamic Data Replication Strategy for Reliability in Cloud Data Centres[C]// 2011 IEEE Ninth International Conference on Dependable. Sydney: IEEE Press, 2011: 496-502.
- [59] Li Wenhao, Yang Yun, Yuan Dong. Ensuring Cloud Data Reliability with Minimum Replication by Proactive Replica Checking[J]. IEEE Transactions on Computers, 2016, 65(5): 1494-1506.
- [60] Ghemawat S, Gobioff H, Leung S T. The Google file system[C]// Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles. New York: ACM Press, 2003: 29-43.
- [61] 谷雷. 面向并行微重启的检查点优化方法[D]. 哈尔滨工程大学, 2017.
- [62] Jassas M, Mahmoud Q H. Failure Analysis and Characterization of Scheduling Jobs in Google Cluster Trace[C]// Annual Conference of the IEEE Industrial Electronics Society. Washington: IEEE Press, 2018: 3102-3107.

致谢

在此落笔之际，表征着我的三年研究生生活即将完结。回首那些白驹过隙的日子，它们由我收到录取通知书时的喜悦、无数个在实验室努力的白天黑夜、遇到困难想放弃却又一次次坚持下来、以及参加阿里算法比赛失利后的失落等等记忆堆砌而成。迷茫过也坚定不移过，有收获成就也有希望落空。但无论这些生活如何跌宕起伏，如何一次次打击我，始终要感谢在背后默默支持和宽慰我的家人、老师、朋友和同学。是你们给了我跨过千难万阻的外因，也感谢自己不服输的精神和万事想得开的心态，这是我走出迷茫、战胜困难的内因。

首先向我的研究生导师何利老师致以最诚挚的感谢和敬意。从 2016 年入学后第一次参加实验室的例会开始，何利老师在这三年时光里至始至终都尽心尽力关心我的生活、学术研究以及个人情感等等方面的情况。不仅对我的成长起到正向引导作用，还教会我在社会中立足的技巧和细节。此外，何利老师还教会我要务实，要有严谨的态度和严密的逻辑思维，既要注重智商发展还要培养情商成长，这些都将使我终生受益。感谢何利老师。

感谢我的好朋友王雪怡对我学业上的帮助和心态的调整。感谢已经毕业的各位师兄和师姐，在我找工作之际提供的帮助和鼓励。同时，也由衷的谢谢实验室团队的各位老师对我的帮助，每位老师对科研工作严谨的态度和钻研精神深深的影响和激励着我，感谢你们对我的批评与指正，帮助及肯定。我还要感谢信科 2012 实验室的小伙伴们，我们共同营造了良好的快乐的学习氛围，高效学习也适度娱乐放松。

另外，我还要感谢 Elsevier 期刊的创办者、LATEX 的开发者、BLCR 框架开发者以及以 Linux 为代表的开源代码贡献者们。是你们让我的能力有了潜移默化的提升，科研界因为你们而变得更好，社会因为你们的努力获得更快速的发展。

最后，要着重感谢我的父母。你们为我付出了无以量计的爱和经济支持。感谢你们支持我，宠爱我，没有你们的严格要求和细致关爱就没有我的今天，谢谢你们。

攻读硕士学位期间从事的科研工作及取得的成果

参与科研项目：

- [1] 云服务模式与度量研究(编号：A2014-21)，博士启动基金科研项目，2014 年 10 月-2017 年 10 月
- [2] 可靠性驱动的移动云资源管理随机模型和优化方法(编号：61602073)，国家自然科学基金，2017 年 1 月-2019 年 12 月
- [3] 移动云服务资源可靠管理机制关键技术研究(编号：cstc2017jcyjA0818)，重庆市基础科学与前沿技术研究项目，2017 年 9 月-2020 年 9 月

发表及完成论文：

- [1] 何利，**曹启彦**. 基于分级检查点的移动云系统容错方法[P]. 中国：申请号：201811185931.8, 2018.10.11, 已受理.
- [2] 何利，**曹启彦**. 基于检验点的移动云资源调度策略研究[J]. 重庆邮电大学学报(自然科学版), 2019, 31(1):107-116.
- [3] Li He, **Qiyao Cao**, Fengjun Shang. A Heuristic Replicas Allocation Strategy in mobile cloud [C]// 2018 IEEE 3rd International Conference on Cloud Computing and Big Data Analysis (ICCCBDA). Chengdu: IEEE press, 2018: 155-159.
- [4] Li He, **Qiyao Cao**, Fengjun Shang. Measuring Component Importance for Network System Using Cellular Automata[J]. Complexity. 2019.